

課程 5.1：std::mutex 基本操作

好的，讓我先查看我們之前的課程進度和大綱。找到了課程大綱。讓我再確認第五階段的完整內容：很好，我確認了課程進度。你已經完成了第四階段（共享資料與競爭條件），現在正式開始第五階段。

第五階段：互斥鎖基礎 (std::mutex)

課程 5.1：std::mutex 基本操作

引言

在第四階段，我們深入了解了競爭條件的危害。現在，我們要學習解決這個問題的第一個武器：**互斥鎖（Mutex）**。Mutex 是 **Mutual Exclusion** 的縮寫，意思是「互相排斥」，確保同一時間只有一個執行緒能存取共享資源。

一、std::mutex 基本概念

互斥鎖（Mutex）概念	
想像一間只能容納一人的更衣室：	
更衣室	← 共享資源（如：全域變數）
🔒 門鎖	← 互斥鎖
規則：	
• 進入前必須鎖門（lock）	
• 離開後必須開門（unlock）	
• 門鎖著時，其他人必須在外等待	

二、std::mutex 的三個基本操作

```
#include <mutex>

std::mutex mtx; // 建立互斥鎖

mtx.lock(); // 🔒 獲取鎖（若已被鎖定，則阻塞等待）
// ... 臨界區段：存取共享資源 ...
mtx.unlock(); // 🔓 釋放鎖
```

方法	功能	阻塞行為
lock()	獲取鎖	若鎖已被持有， 阻塞等待 直到獲取成功
unlock()	釋放鎖	非阻塞
try_lock()	嘗試獲取鎖	非阻塞 ，立即返回成功或失敗

三、解決競爭條件：完整範例

問題回顧（無保護的程式碼）

```
// 檔案：lesson_5_1_no_mutex.cpp
// 說明：沒有互斥鎖保護的計數器（有競爭條件）

#include <iostream>
#include <thread>

int counter = 0; // 共享資源

void increment() {
    for (int i = 0; i < 100000; ++i) {
        ++counter; // 危險！非原子操作
    }
}

int main() {
```

```

std::thread t1(increment);
std::thread t2(increment);
t1.join();
t2.join();
std::cout << "預期: 200000" << std::endl;
std::cout << "實際: " << counter << std::endl; // 通常小於 200000
return 0;
}

```

編譯與執行

```

g++ -std=c++17 -pthread -o no_mutex lesson_5_1_no_mutex.cpp
./no_mutex

```

可能的輸出

```

預期: 200000
實際: 143287

```

解決方案（使用 std::mutex）

```

// 檔案: lesson_5_1_with_mutex.cpp
// 說明: 使用互斥鎖保護共享資源

#include <iostream>
#include <thread>
#include <mutex>

int counter = 0; // 共享資源
std::mutex mtx; // 保護 counter 的互斥鎖

void increment() {
    for (int i = 0; i < 100000; ++i) {
        mtx.lock(); // 🔒 進入臨界區段前獲取鎖
        ++counter; // 安全! 一次只有一個執行緒能執行
        mtx.unlock(); // 🔓 離開臨界區段後釋放鎖
    }
}

```

```

}

int main() {
    std::thread t1(increment);
    std::thread t2(increment);
    t1.join();
    t2.join();
    std::cout << "預期: 200000" << std::endl;
    std::cout << "實際: " << counter << std::endl; // 保證是 200000
    return 0;
}

```

編譯與執行

```

g++ -std=c++17 -pthread -o with_mutex lesson_5_1_with_mutex.cpp
./with_mutex

```

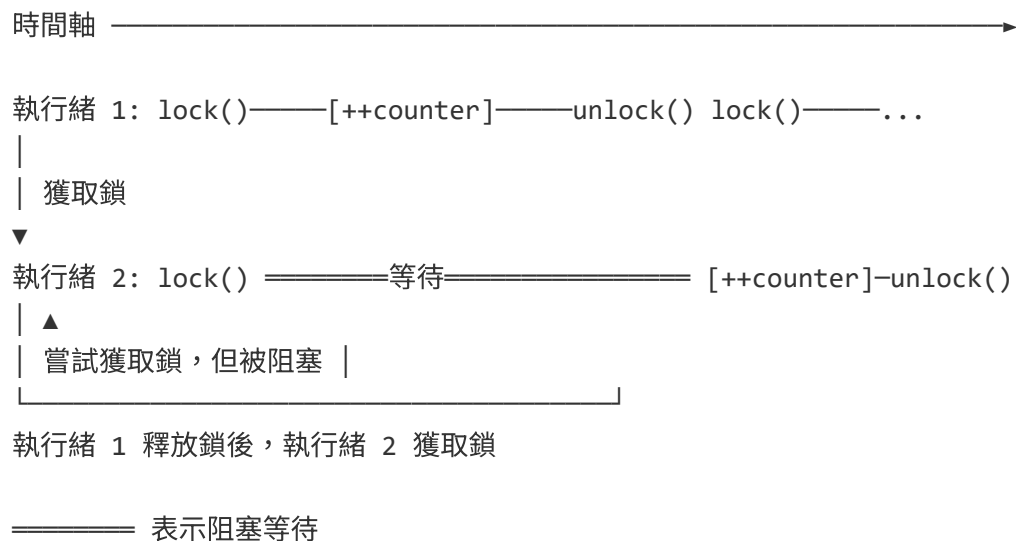
輸出

```

預期: 200000
實際: 200000

```

四、執行流程圖解



—— 表示持有鎖執行

五、多個操作的保護

當臨界區段包含多個操作時：

```
// 檔案：lesson_5_1_multiple_ops.cpp
// 說明：保護多個相關操作

#include <iostream>
#include <thread>
#include <mutex>
#include <vector>

std::vector<int> data;
std::mutex mtx;

void addAndPrint(int value) {
    mtx.lock();
    // 以下三個操作必須作為一個整體執行
    data.push_back(value); // 操作 1：新增元素
    std::cout << "Added: " << value; // 操作 2：輸出訊息
    std::cout << ", Size: " << data.size(); // 操作 3：輸出大小
    std::cout << std::endl;
    mtx.unlock();
}

int main() {
    std::thread t1([]() {
        for (int i = 0; i < 5; ++i) {
            addAndPrint(i);
        }
    });
    std::thread t2([]() {
        for (int i = 100; i < 105; ++i) {
            addAndPrint(i);
        }
    });
    t1.join();
```

```
t2.join();
return 0;
}
```

輸出（順序可能不同，但每行是完整的）

```
Added: 0, Size: 1
Added: 100, Size: 2
Added: 1, Size: 3
Added: 101, Size: 4
Added: 2, Size: 5
...
```

若沒有互斥鎖，輸出可能會交錯混亂：

```
Added: 0Added: 100, Size: , Size: 21
```

六、lock() 與 unlock() 的配對規則

配對規則（重要！）	
	✓ 正確：每個 lock() 都有對應的 unlock()
	mtx.lock();
	// 工作
	mtx.unlock(); // ✓ 配對
	X 錯誤 1：忘記 unlock()
	mtx.lock();
	// 工作
	// 沒有 unlock() → 其他執行緒永遠等待（死結）
	X 錯誤 2：重複 lock()（同一執行緒）

```
| mtx.lock(); |  
| mtx.lock(); // 💀 未定義行為！（標準 mutex 不支援） |  
| |  
| X 錯誤 3：unlock() 未鎖定的互斥鎖 |  
| |  
| mtx.unlock(); // 💀 未定義行為！ |  
| |  
|_|
```

七、忘記 unlock() 的危險

// 檔案：lesson_5_1_forget_unlock.cpp

// 說明：忘記 unlock() 導致的問題（示範用，不要在實際專案這樣寫）

```
#include <iostream>
```

```
#include <thread>
```

```
#include <mutex>
```

```
std::mutex mtx;
```

```
void badFunction() {
```

```
    mtx.lock();
```

```
    std::cout << "Thread " << std::this_thread::get_id() << " acquired lock\n";
```

```
    // 假設這裡發生早期返回或拋出例外
```

```
    if (true) {
```

```
        return; // 💀 忘記 unlock() !
```

```
    }
```

```
    mtx.unlock(); // 永遠不會執行
```

```
}
```

```
int main() {
```

```
    std::thread t1(badFunction);
```

```
    t1.join();
```

```
    std::cout << "Thread 1 finished\n";
```

```
    std::thread t2(badFunction); // 💀 永遠卡住！因為鎖沒被釋放
```

```
    t2.join();
```

```
    std::cout << "Thread 2 finished\n"; // 永遠不會執行
```

```
    return 0;
```

```
}
```

這就是為什麼我們需要 RAII 鎖管理器（下一階段會學習）。

八、std::mutex 的特性

std::mutex 特性
不可複製、不可移動
std::mutex mtx1;
std::mutex mtx2 = mtx1; // X 編譯錯誤
不支援遞迴鎖定
同一執行緒 lock() 兩次 → 未定義行為
（需要遞迴鎖定时，使用 std::recursive_mutex）
lock() 可能拋出例外
std::system_error（如：資源不足）
預設建構，無需參數
std::mutex mtx; // 即可使用

九、本課重點回顧

1. **std::mutex** 是最基本的同步機制，確保臨界區段的互斥存取
2. 三個基本操作：lock()、unlock()、try_lock()
3. lock() 會阻塞等待直到獲取鎖
4. 每個 lock() 必須有對應的 unlock()
5. 忘記 unlock() 會導致其他執行緒永遠等待
6. std::mutex 不支援同一執行緒重複鎖定

十、練習題

練習 1：修改以下程式碼，使用 std::mutex 解決競爭條件


```
#include <iostream>
#include <thread>

int balance = 1000;

void withdraw(int amount) {
    if (balance >= amount) {
        balance -= amount;
        std::cout << "Withdrew: " << amount << ", Balance: " << balance << std::endl;
    }
}

int main() {
    std::thread t1(withdraw, 800);
    std::thread t2(withdraw, 800);
    t1.join();
    t2.join();
    std::cout << "Final balance: " << balance << std::endl;
    return 0;
}
```

下一課預告

在 **課程 5.2：互斥鎖的工作原理** 中，我們將深入了解：

- 互斥鎖在作業系統層面如何運作
- 自旋鎖 vs 睡眠鎖
- 鎖的獲取與釋放過程

準備好繼續嗎？

課程 5.2：互斥鎖的工作原理

第五階段：互斥鎖基礎 (std::mutex)

課程 5.2：互斥鎖的工作原理

引言

在上一課，我們學會了如何使用 `lock()` 和 `unlock()` 來保護共享資料。但互斥鎖內部究竟是如何運作的？為什麼它能確保同一時間只有一個執行緒進入臨界區段？本課將揭開互斥鎖的神秘面紗。

一、互斥鎖的核心概念

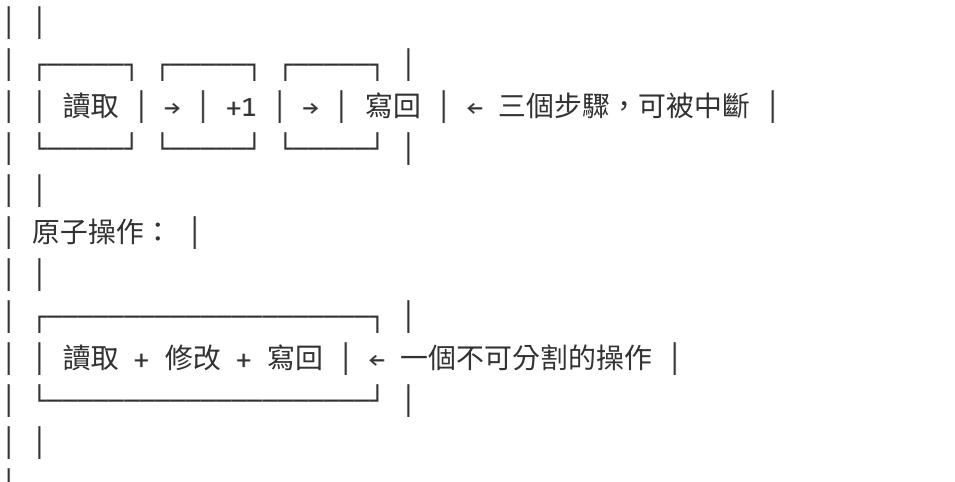
互斥鎖的本質	
互斥鎖本質上是一個「狀態變數」+「等待機制」的組合：	
Mutex 內部狀態	- 鎖定狀態 (<code>locked</code> / <code>unlocked</code>) - 擁有者 (哪個執行緒持有鎖) - 等待佇列 (等待獲取鎖的執行緒)
關鍵：狀態的改變必須是「原子操作」	

二、原子操作：互斥鎖的基石

互斥鎖依賴 CPU 提供的**原子指令**來確保狀態改變不會被中斷。

什麼是原子操作？

原子操作	
原子 (Atomic) = 不可分割	
非原子操作 (如 <code>counter++</code>):	



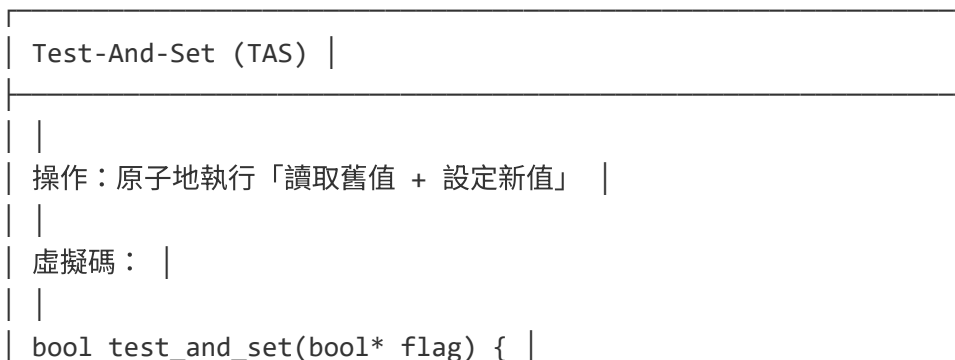
CPU 層級的原子指令

現代 CPU 提供特殊指令來實現原子操作：

指令	架構	功能
LOCK CMPXCHG	x86/x64	Compare-And-Swap (CAS)
LOCK XCHG	x86/x64	原子交換
LDREX/STREX	ARM (舊)	Load-Exclusive / Store-Exclusive
LDXR/STXR	ARM (新)	原子載入/儲存

三、Test-And-Set：最簡單的鎖實現

概念說明



```

| bool old = *flag; //  $\neg$  |
| *flag = true; //  $\perp$  這兩步是原子的 |
| return old; |
| } |
| |
| 使用方式： |
| |
| while (test_and_set(&locked)) { |
| // 自旋等待 |
| } |
| // 獲得鎖，進入臨界區段 |
| |

```

簡化的自旋鎖實現（概念示範）

// 檔案：lesson_5_2_spinlock_concept.cpp
// 說明：展示自旋鎖的基本概念（僅供理解，非生產環境使用）

```

#include <iostream>
#include <thread>
#include <atomic>
#include <vector>

class SimpleSpinlock {
private:
    std::atomic<bool> locked{false};
public:
    void lock() {
        // exchange 是原子操作：設定新值並返回舊值
        while (locked.exchange(true)) {
            // 如果舊值是 true，表示已被鎖定，繼續自旋
            // 這裡可以加入 yield 或 pause 來減少 CPU 消耗
        }
        // 如果舊值是 false，表示成功獲取鎖
    }
    void unlock() {
        locked.store(false);
    }
};

```

```
// 測試
SimpleSpinlock spinlock;
int counter = 0;

void increment() {
    for (int i = 0; i < 100000; ++i) {
        spinlock.lock();
        ++counter;
        spinlock.unlock();
    }
}

int main() {
    std::thread t1(increment);
    std::thread t2(increment);
    t1.join();
    t2.join();
    std::cout << "預期: 200000" << std::endl;
    std::cout << "實際: " << counter << std::endl;
    return 0;
}
```

編譯與執行

```
g++ -std=c++17 -pthread -o spinlock lesson_5_2_spinlock_concept.cpp
./spinlock
```

輸出

```
預期: 200000
實際: 200000
```

四、Compare-And-Swap (CAS)：更強大的原子操作

CAS 操作原理

Compare-And-Swap (CAS)
操作：比較並交換 <pre> bool CAS(int* ptr, int expected, int desired) { if (*ptr == expected) { // ㄣ *ptr = desired; // 整個操作是原子的 return true; // } // ㄣ return false; } </pre> 意義：「只有當前值符合預期時，才更新」

CAS 實現鎖的範例

// 檔案：lesson_5_2_cas_lock.cpp
 // 說明：使用 CAS 實現的自旋鎖

```

#include <iostream>
#include <thread>
#include <atomic>

class CASSpinlock {
private:
    std::atomic<bool> locked{false};
public:
    void lock() {
        bool expected = false;
        // compare_exchange_strong:
        // 如果 locked == expected(false)，則設為 true，返回 true
        // 如果 locked != expected，則 expected 被更新為 locked 的值，返回 false
        while (!locked.compare_exchange_strong(expected, true)) {
            expected = false; // 重設 expected 為 false
            // 繼續嘗試
        }
    }
}

```

```

void unlock() {
    locked.store(false);
}
};

CASSpinlock cas_lock;
int shared_counter = 0;

void worker() {
    for (int i = 0; i < 100000; ++i) {
        cas_lock.lock();
        ++shared_counter;
        cas_lock.unlock();
    }
}

int main() {
    std::thread t1(worker);
    std::thread t2(worker);
    t1.join();
    t2.join();
    std::cout << "counter = " << shared_counter << std::endl;
    return 0;
}

```

五、自旋鎖 vs 睡眠鎖

自旋鎖 vs 睡眠鎖
自旋鎖 (Spinlock) • 等待時不斷循環檢查鎖狀態 • 持續佔用 CPU • 適合：鎖持有時間極短的情況 while (locked) {

```
| // 忙等待，消耗 CPU |
| } |
| |
| _____ |
| |
| 睡眠鎖 (Sleeping Lock / Blocking Lock) |
| _____ |
| • 等待時讓出 CPU，進入睡眠狀態 |
| • 需要作業系統介入（系統呼叫） |
| • 適合：鎖持有時間較長的情況 |
| |
| if (locked) { |
| 加入等待佇列; |
| 睡眠; // 讓出 CPU |
| } |
| |
|_____|
```

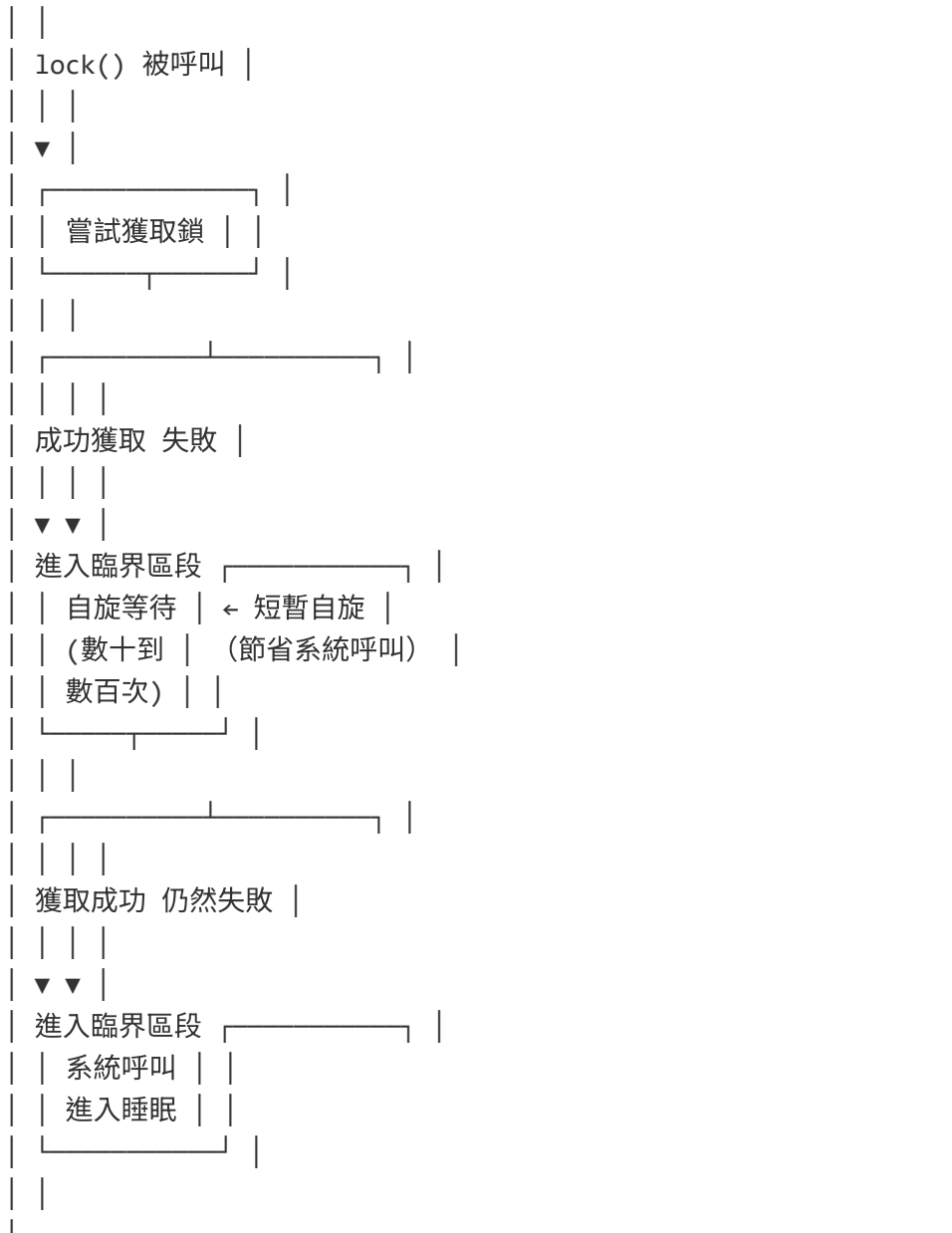
比較表

特性	自旋鎖	睡眠鎖
等待方式	忙等待 (busy-wait)	睡眠 (block)
CPU 消耗	高 (持續佔用)	低 (讓出 CPU)
上下文切換	無	有 (開銷較大)
適用場景	臨界區段極短	臨界區段較長
適用環境	多核心系統	任何環境

六、std::mutex 的實際實現

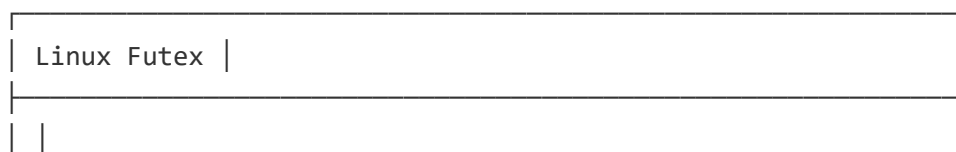
std::mutex 通常採用**混合策略**：先自旋一小段時間，若仍無法獲取鎖，再進入睡眠。

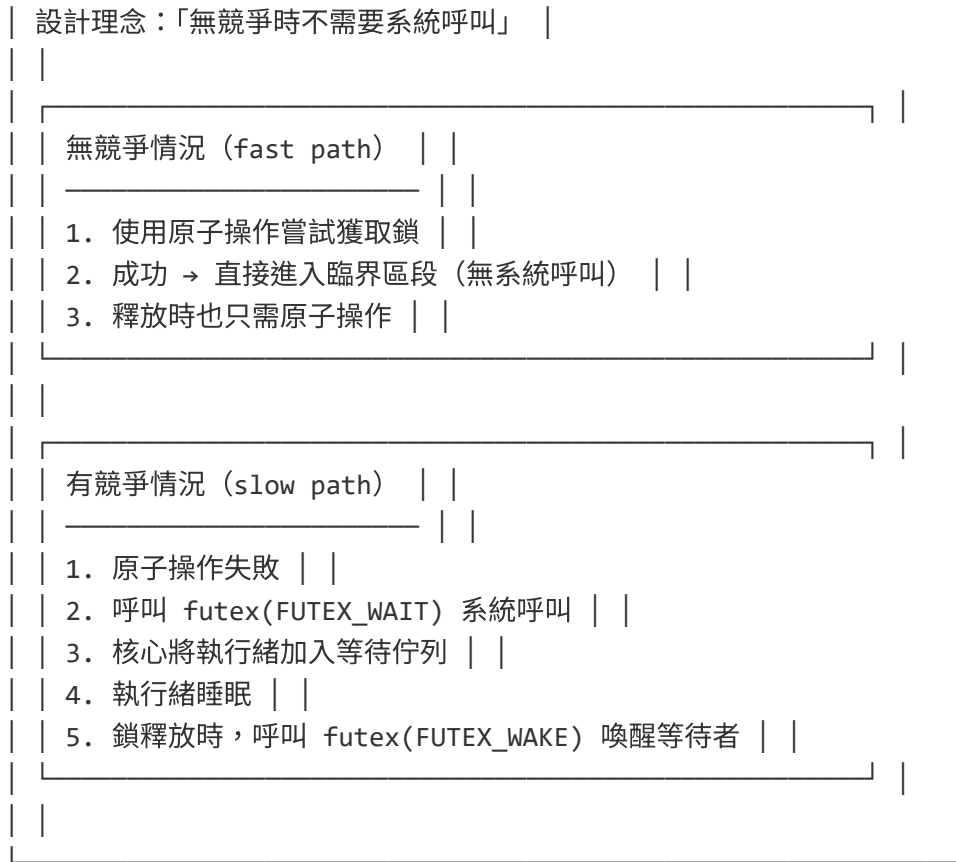
```
|_____|
| std::mutex 的混合策略 |
|_____|
```

七、作業系統層面的實現

Linux : Futex (Fast Userspace Mutex)





簡化的 Futex 概念示範

```
// 檔案：lesson_5_2_futex_concept.cpp
// 說明：Futex 概念的簡化說明（非實際實現）

#include <iostream>
#include <atomic>

/*
 * 這是 Futex 的概念說明，不是實際可編譯的程式碼
 *
 * Futex 的核心思想：
 *
 * struct futex_mutex {
 * atomic<int> state; // 0 = unlocked, 1 = locked (無等待者), 2 = locked (有等待者)
 * };
 *
```

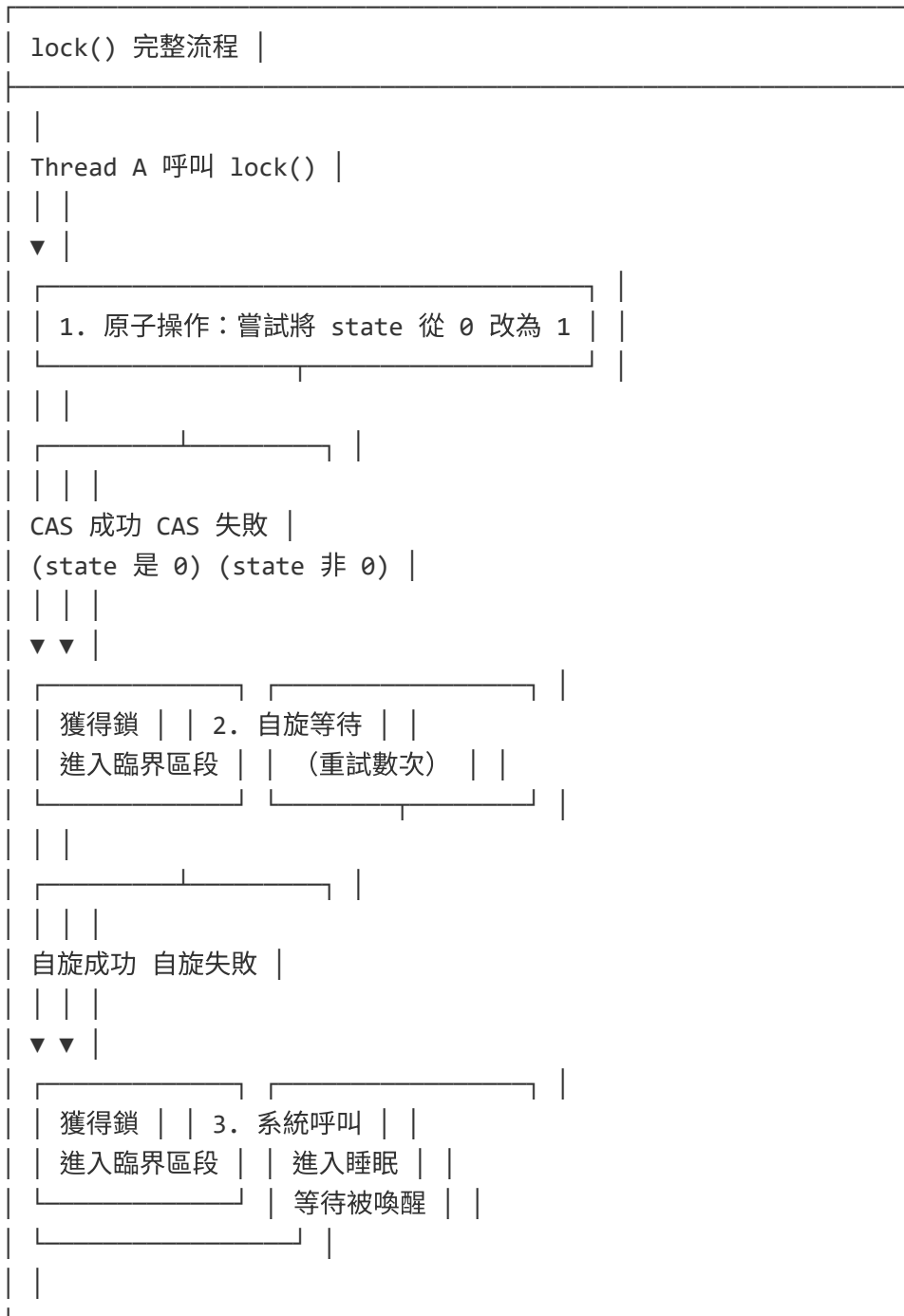
```

* void lock() {
*   int expected = 0;
*
*   // Fast path: 嘗試 0 → 1 (無競爭)
*   if (state.compare_exchange_strong(expected, 1)) {
*     return; // 成功獲取，無需系統呼叫
*   }
*
*   // Slow path: 有競爭
*   while (true) {
*     // 標記為「有等待者」
*     if (expected == 2 || state.compare_exchange_strong(expected, 2)) {
*       // 系統呼叫：等待 state 變化
*       futex_wait(&state, 2);
*     }
*
*     expected = 0;
*     if (state.compare_exchange_strong(expected, 2)) {
*       return; // 成功獲取
*     }
*   }
*
*   void unlock() {
*     // 如果 state 從 1 變成 0，無等待者
*     if (state.fetch_sub(1) == 1) {
*       return; // 無需喚醒
*     }
*
*     // 有等待者，需要喚醒
*     state.store(0);
*     futex_wake(&state, 1); // 喚醒一個等待者
*   }
* }

int main() {
std::cout << "Futex 是 Linux 實現高效互斥鎖的機制" << std::endl;
std::cout << "無競爭時：純用戶空間原子操作，無系統呼叫" << std::endl;
std::cout << "有競爭時：使用系統呼叫讓執行緒睡眠" << std::endl;
return 0;

```

八、lock() 的完整流程

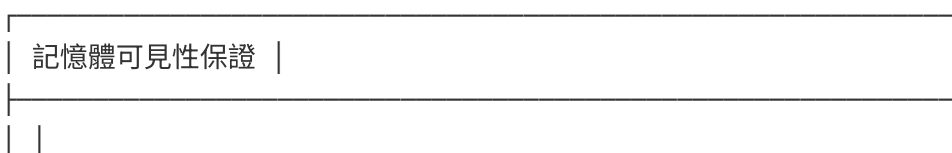


九、unlock() 的完整流程



十、記憶體屏障與可見性

互斥鎖不僅僅是「排他存取」，還提供**記憶體可見性保證**。



```

| 問題：現代 CPU 有快取、指令重排 |
| |
| Thread A Thread B |
| _____ |
| data = 42; |
| ready = true; if (ready) { |
| use(data); // data 是? |
| } |
| |
| 沒有同步機制時，Thread B 可能看到 ready=true 但 data≠42 |
| |
| _____ |
| |
| 互斥鎖的保證： |
| |
| Thread A Thread B |
| _____ |
| mtx.lock(); |
| data = 42; |
| mtx.unlock(); —————→ mtx.lock(); |
| | use(data); // 保證是 42 |
| | mtx.unlock(); |
| | |
| ↳ unlock() 包含「釋放屏障」(release barrier) |
| lock() 包含「獲取屏障」(acquire barrier) |
| |
| _____ |

```

記憶體屏障的作用

```

// 檔案：lesson_5_2_memory_barrier.cpp
// 說明：互斥鎖提供的記憶體可見性保證

```

```

#include <iostream>
#include <thread>
#include <mutex>

std::mutex mtx;
int data = 0;
bool ready = false;

```

```

void producer() {
    mtx.lock();
    data = 42; // 寫入資料
    ready = true; // 設定標誌
    mtx.unlock(); // unlock 包含 release barrier
    // 保證 data 和 ready 的寫入對其他執行緒可見
}

void consumer() {
    mtx.lock(); // lock 包含 acquire barrier
    // 保證看到 producer 的所有寫入
    if (ready) {
        std::cout << "data = " << data << std::endl; // 保證是 42
    }
    mtx.unlock();
}

int main() {
    std::thread t1(producer);
    std::thread t2(consumer);
    t1.join();
    t2.join();
    return 0;
}

```

十一、效能考量

互斥鎖的效能開銷	
無競爭時 (Fast Path):	
• 約 10-25 奈秒 (一次原子操作 + 記憶體屏障)	
• 純用戶空間，無系統呼叫	
有競爭時 (Slow Path):	
• 自旋階段：數百奈秒	

- 若需睡眠：數微秒（系統呼叫開銷）
- 上下文切換：1-10 微秒

優化建議：

1. 減少臨界區段的大小
2. 避免在臨界區段內進行 I/O 操作
3. 考慮使用細粒度鎖或讀寫鎖
4. 高競爭場景考慮無鎖演算法

十二、本課重點回顧

1. 互斥鎖依賴原子操作：如 Test-And-Set、Compare-And-Swap
2. 自旋鎖：忙等待，適合極短臨界區段
3. 睡眠鎖：讓出 CPU，適合較長臨界區段
4. `std::mutex` 採用混合策略：先自旋，再睡眠
5. Linux 使用 `Futex`：無競爭時無系統呼叫
6. 互斥鎖提供記憶體可見性保證：包含記憶體屏障
7. 無競爭時開銷約 10-25 奈秒，有競爭時可達微秒級

下一課預告

在 課程 5.3：try_lock() 非阻塞鎖定 中，我們將學習：

- 如何嘗試獲取鎖而不阻塞
- try_lock() 的返回值處理
- 非阻塞鎖定的應用場景

準備好繼續嗎？

課程 5.3：try_lock() 非阻塞鎖定

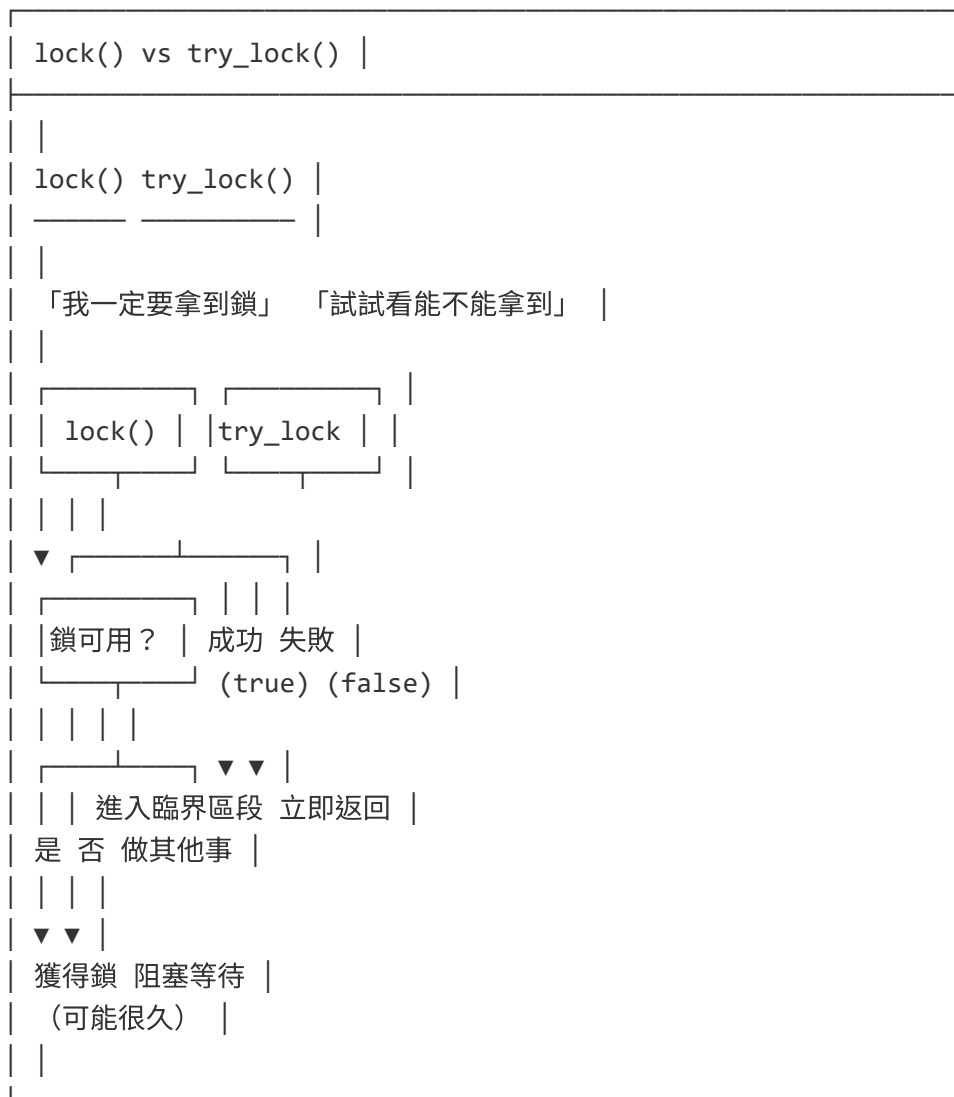
第五階段：互斥鎖基礎 (std::mutex)

課程 5.3：try_lock() 非阻塞鎖定

引言

在前兩課中，我們學習了 `lock()` 會阻塞等待直到獲取鎖。但有時候，我們不希望執行緒傻傻地等待，而是想「試試看能不能拿到鎖，拿不到就做別的事」。這就是 `try_lock()` 的用途。

一、lock() vs try_lock()



方法	行為	返回值	阻塞
lock()	獲取鎖，失敗則等待	無 (void)	是
try_lock()	嘗試獲取鎖，失敗則立即返回	true / false	否
unlock()	釋放鎖	無 (void)	否

二、try_lock() 基本用法

// 檔案：lesson_5_3_try_lock_basic.cpp
 // 說明：try_lock() 的基本使用方式

```
#include <iostream>
#include <thread>
#include <mutex>
#include <chrono>

std::mutex mtx;

void worker(int id) {
  // 嘗試獲取鎖
  if (mtx.try_lock()) {
    // 成功獲取鎖
    std::cout << "執行緒 " << id << "：成功獲取鎖，開始工作..." << std::endl;
    // 模擬工作
    std::this_thread::sleep_for(std::chrono::milliseconds(100));
    std::cout << "執行緒 " << id << "：工作完成，釋放鎖" << std::endl;
    mtx.unlock(); // 記得解鎖！
  } else {
    // 獲取失敗，立即返回
    std::cout << "執行緒 " << id << "：無法獲取鎖，執行其他任務" << std::endl;
  }
}

int main() {
  std::thread t1(worker, 1);
  std::thread t2(worker, 2);
```

```
std::thread t3(worker, 3);  
t1.join();  
t2.join();  
t3.join();  
return 0;  
}
```

編譯與執行

```
g++ -std=c++17 -pthread -o try_lock_basic lesson_5_3_try_lock_basic.cpp  
./try_lock_basic
```

可能的輸出

執行緒 1：成功獲取鎖，開始工作...
執行緒 2：無法獲取鎖，執行其他任務
執行緒 3：無法獲取鎖，執行其他任務
執行緒 1：工作完成，釋放鎖

三、重複嘗試模式

有時我們想重複嘗試，但不想無限等待：

```
// 檔案：lesson_5_3_retry_pattern.cpp  
// 說明：使用 try_lock() 實現有限次數的重試  
  
#include <iostream>  
#include <thread>  
#include <mutex>  
#include <chrono>  
  
std::mutex mtx;  
int shared_data = 0;  
  
bool tryUpdateWithRetry(int id, int maxRetries) {  
    for (int attempt = 1; attempt <= maxRetries; ++attempt) {  
        if (mtx.try_lock()) {  
            // 成功獲取鎖
```

```

std::cout << "執行緒 " << id << "：第 " << attempt
<< " 次嘗試成功" << std::endl;
++shared_data;
// 模擬一些工作
std::this_thread::sleep_for(std::chrono::milliseconds(50));
mtx.unlock();
return true;
}
std::cout << "執行緒 " << id << "：第 " << attempt
<< " 次嘗試失敗，稍後重試..." << std::endl;
// 等待一小段時間再重試
std::this_thread::sleep_for(std::chrono::milliseconds(10));
}
std::cout << "執行緒 " << id << "：達到最大重試次數，放棄" << std::endl;
return false;
}

int main() {
std::thread t1(tryUpdateWithRetry, 1, 5);
std::thread t2(tryUpdateWithRetry, 2, 5);
t1.join();
t2.join();
std::cout << "最終 shared_data = " << shared_data << std::endl;
return 0;
}

```

可能的輸出

```

執行緒 1：第 1 次嘗試成功
執行緒 2：第 1 次嘗試失敗，稍後重試...
執行緒 2：第 2 次嘗試失敗，稍後重試...
執行緒 2：第 3 次嘗試失敗，稍後重試...
執行緒 2：第 4 次嘗試失敗，稍後重試...
執行緒 2：第 5 次嘗試失敗，稍後重試...
執行緒 1：(解鎖)
執行緒 2：第 5 次嘗試成功
最終 shared_data = 2

```

四、避免死結的應用

`try_lock()` 最重要的應用之一是避免死結。

死結情境回顧

死結情境	
Thread A Thread B	

lock(mutex1); lock(mutex2);	
lock(mutex2); ← 等待 B lock(mutex1); ← 等待 A	
互相等待 → 死結！	

使用 `try_lock()` 避免死結

// 檔案：lesson_5_3_avoid_deadlock.cpp

// 說明：使用 `try_lock()` 避免死結

```
#include <iostream>
#include <thread>
#include <mutex>
#include <chrono>

std::mutex mutex1;
std::mutex mutex2;

void safeOperation(int id, std::mutex& first, std::mutex& second,
const char* firstName, const char* secondName) {
while (true) {
// 先鎖定第一個互斥鎖
first.lock();
std::cout << "執行緒 " << id << "：獲得 " << firstName << std::endl;
// 嘗試鎖定第二個互斥鎖
if (second.try_lock()) {
// 成功獲得兩個鎖
std::cout << "執行緒 " << id << "：獲得 " << secondName
```

```

<< "，執行操作" << std::endl;
// 執行需要兩個鎖的操作
std::this_thread::sleep_for(std::chrono::milliseconds(50));
// 釋放鎖（注意順序：後獲取的先釋放）
second.unlock();
first.unlock();
std::cout << "執行緒 " << id << "：操作完成" << std::endl;
return; // 成功完成
}
// 無法獲得第二個鎖，釋放第一個鎖，避免死結
std::cout << "執行緒 " << id << "：無法獲得 " << secondName
<< "，釋放 " << firstName << " 並重試" << std::endl;
first.unlock();
// 稍等一下再重試（避免活鎖）
std::this_thread::sleep_for(std::chrono::milliseconds(1));
}
}

int main() {
// 兩個執行緒以不同順序請求鎖
std::thread t1([](){
safeOperation(1, mutex1, mutex2, "mutex1", "mutex2");
});
std::thread t2([](){
safeOperation(2, mutex2, mutex1, "mutex2", "mutex1");
});
t1.join();
t2.join();
std::cout << "兩個執行緒都成功完成，沒有死結！" << std::endl;
return 0;
}

```

輸出

```

執行緒 1：獲得 mutex1
執行緒 2：獲得 mutex2
執行緒 1：無法獲得 mutex2，釋放 mutex1 並重試
執行緒 2：無法獲得 mutex1，釋放 mutex2 並重試
執行緒 1：獲得 mutex1
執行緒 1：獲得 mutex2，執行操作

```

執行緒 1：操作完成
執行緒 2：獲得 mutex2
執行緒 2：獲得 mutex1，執行操作
執行緒 2：操作完成
兩個執行緒都成功完成，沒有死結！

五、活鎖（Livelock）問題

```
| 活鎖警告！ |  
|  
| 使用 try_lock() 可能導致「活鎖」： |  
|  
| Thread A Thread B |  
| _____ |  
| lock(m1) lock(m2) |  
| try m2 失敗 try m1 失敗 |  
| unlock(m1) unlock(m2) |  
| lock(m1) lock(m2) ← 立刻重試 |  
| try m2 失敗 try m1 失敗 ← 又失敗 |  
| ... ... ← 無限循環！ |  
|  
| 解決方案：加入隨機延遲 |  
|  
| // 重試前等待隨機時間 |  
| std::this_thread::sleep_for( |  
| std::chrono::milliseconds(rand() % 10) |  
| ); |  
| |
```

避免活鎖的改進版本

```
// 檔案：lesson_5_3_avoid_livelock.cpp  
// 說明：使用隨機退避避免活鎖
```

```

#include <iostream>
#include <thread>
#include <mutex>
#include <chrono>
#include <random>

std::mutex mutex1;
std::mutex mutex2;

void robustOperation(int id) {
    // 隨機數產生器
    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_int_distribution<> dis(1, 10);
    int attempts = 0;
    const int maxAttempts = 100;
    while (attempts < maxAttempts) {
        ++attempts;
        mutex1.lock();
        if (mutex2.try_lock()) {
            // 成功獲得兩個鎖
            std::cout << "執行緒 " << id << "：成功（嘗試 "
            << attempts << " 次）" << std::endl;
            // 執行操作
            std::this_thread::sleep_for(std::chrono::milliseconds(10));
            mutex2.unlock();
            mutex1.unlock();
            return;
        }
        // 釋放並等待隨機時間（指數退避的簡化版）
        mutex1.unlock();
        int waitTime = dis(gen); // 1-10 毫秒的隨機等待
        std::this_thread::sleep_for(std::chrono::milliseconds(waitTime));
    }
    std::cout << "執行緒 " << id << "：達到最大嘗試次數" << std::endl;
}

int main() {
    std::thread t1(robustOperation, 1);
    std::thread t2(robustOperation, 2);

```



```
t2.join();  
return 0;  
}
```

六、實用場景：非阻塞資源存取

```
// 檔案：lesson_5_3_non_blocking_cache.cpp  
// 說明：非阻塞的快取存取模式
```

```
#include <iostream>  
#include <thread>  
#include <mutex>  
#include <map>  
#include <string>  
#include <chrono>  
#include <vector>  
  
class NonBlockingCache {  
private:  
    std::map<std::string, std::string> cache;  
    std::mutex mtx;  
public:  
    // 嘗試從快取獲取值（非阻塞）  
    bool tryGet(const std::string& key, std::string& value) {  
        if (mtx.try_lock()) {  
            auto it = cache.find(key);  
            if (it != cache.end()) {  
                value = it->second;  
                mtx.unlock();  
                return true;  
            }  
            mtx.unlock();  
        }  
        // 無法獲取鎖或找不到 key  
        return false;  
    }  
    // 嘗試設定值（非阻塞）  
    bool trySet(const std::string& key, const std::string& value) {
```

```

    if (mtx.try_lock()) {
        cache[key] = value;
        mtx.unlock();
        return true;
    }
    return false;
}
// 阻塞版本 (保證成功)
void set(const std::string& key, const std::string& value) {
    mtx.lock();
    cache[key] = value;
    mtx.unlock();
}
std::string get(const std::string& key) {
    mtx.lock();
    std::string result = cache[key];
    mtx.unlock();
    return result;
}
};

NonBlockingCache cache;

void fastReader(int id) {
    std::string value;
    for (int i = 0; i < 5; ++i) {
        if (cache.tryGet("data", value)) {
            std::cout << "Reader " << id << "：快速讀取成功 = " << value << std::endl;
        } else {
            std::cout << "Reader " << id << "：快取忙碌，使用預設值" << std::endl;
            // 使用預設值或其他邏輯
        }
        std::this_thread::sleep_for(std::chrono::milliseconds(10));
    }
}

void writer() {
    for (int i = 0; i < 5; ++i) {
        std::string value = "value_" + std::to_string(i);
        cache.set("data", value);
    }
}

```

```

std::this_thread::sleep_for(std::chrono::milliseconds(30));
}
}

int main() {
    cache.set("data", "initial");
    std::thread w(writer);
    std::thread r1(fastReader, 1);
    std::thread r2(fastReader, 2);
    w.join();
    r1.join();
    r2.join();
    return 0;
}

```

七、try_lock() 的注意事項

try_lock() 注意事項
1. 成功時必須 unlock() <pre> if (mtx.try_lock()) { // 工作... mtx.unlock(); // ← 不要忘記！ } </pre>
2. 可能產生虛假失敗 (Spurious Failure) 即使鎖是空閒的，try_lock() 也可能返回 false (實作相關，較少見)
3. 不保證公平性 頻繁呼叫 try_lock() 的執行緒可能一直搶不到鎖
4. 不適合替代 lock()

```
| _____ |  
| 如果一定要獲取鎖，就用 lock() |  
| try_lock() 是用於「可以不獲取」的場景 |  
| |  
|_____|
```

八、try_lock() 與 RAII

手動管理 try_lock() 容易忘記 unlock()，可以搭配 std::unique_lock 使用：

```
// 檔案：lesson_5_3_try_lock_raii.cpp  
// 說明：使用 unique_lock 搭配 try_lock  
  
#include <iostream>  
#include <thread>  
#include <mutex>  
  
std::mutex mtx;  
int counter = 0;  
  
void safeIncrement(int id) {  
    // std::try_to_lock 讓 unique_lock 使用 try_lock() 而非 lock()  
    std::unique_lock<std::mutex> lock(mtx, std::try_to_lock);  
    if (lock.owns_lock()) {  
        // 成功獲取鎖  
        ++counter;  
        std::cout << "執行緒 " << id << "：成功遞增，counter = "  
        << counter << std::endl;  
        // lock 離開作用域時自動 unlock  
    } else {  
        // 獲取失敗  
        std::cout << "執行緒 " << id << "：無法獲取鎖" << std::endl;  
    }  
    // 無論成功與否，都不需要手動 unlock  
}  
  
int main() {  
    std::thread t1(safeIncrement, 1);  
    std::thread t2(safeIncrement, 2);
```

```

std::thread t3(safeIncrement, 3);
t1.join();
t2.join();
t3.join();
std::cout << "最終 counter = " << counter << std::endl;
return 0;
}

```

九、何時使用 try_lock()

try_lock() 適用場景
✓ 適合使用 try_lock() 的情況： • 有替代方案時（如使用預設值、快取） • 避免死結時（嘗試多個鎖） • 實現超時機制時 • 輪詢式設計時（定期檢查資源） • 效能敏感且可接受偶爾失敗時
✗ 不適合使用 try_lock() 的情況： • 操作必須完成時 • 沒有合理的失敗處理邏輯時 • 會導致活鎖時（頻繁重試）

十、完整範例：帶超時的鎖獲取

```

// 檔案：lesson_5_3_try_lock_timeout.cpp
// 說明：實現自訂的超時鎖獲取

```

```

#include <iostream>
#include <thread>
#include <mutex>
#include <chrono>

std::mutex mtx;

// 嘗試在指定時間內獲取鎖
bool tryLockFor(std::mutex& m, std::chrono::milliseconds timeout) {
    auto start = std::chrono::steady_clock::now();
    while (true) {
        if (m.try_lock()) {
            return true; // 成功獲取
        }
        auto elapsed = std::chrono::steady_clock::now() - start;
        if (elapsed >= timeout) {
            return false; // 超時
        }
        // 短暫睡眠避免忙等待
        std::this_thread::sleep_for(std::chrono::milliseconds(1));
    }
}

void worker(int id, int workTime) {
    std::cout << "執行緒 " << id << "：嘗試獲取鎖（超時 100ms）" << std::endl;
    if (tryLockFor(mtx, std::chrono::milliseconds(100))) {
        std::cout << "執行緒 " << id << "：獲得鎖，工作 "
        << workTime << "ms" << std::endl;
        std::this_thread::sleep_for(std::chrono::milliseconds(workTime));
        mtx.unlock();
        std::cout << "執行緒 " << id << "：完成並釋放鎖" << std::endl;
    } else {
        std::cout << "執行緒 " << id << "：超時，放棄等待" << std::endl;
    }
}

int main() {
    // 執行緒 1 持有鎖 200ms，執行緒 2 和 3 只等待 100ms
    std::thread t1(worker, 1, 200);
    std::this_thread::sleep_for(std::chrono::milliseconds(10));
}

```

```
std::thread t3(worker, 3, 50);  
t1.join();  
t2.join();  
t3.join();  
return 0;  
}
```

輸出

執行緒 1：嘗試獲取鎖（超時 100ms）
執行緒 1：獲得鎖，工作 200ms
執行緒 2：嘗試獲取鎖（超時 100ms）
執行緒 3：嘗試獲取鎖（超時 100ms）
執行緒 2：超時，放棄等待
執行緒 3：超時，放棄等待
執行緒 1：完成並釋放鎖

十一、本課重點回顧

1. **try_lock() 是非阻塞的**：立即返回 true（成功）或 false（失敗）
2. **成功時必須 unlock()**：與 lock() 相同的責任
3. **可用於避免死結**：嘗試獲取多個鎖，失敗就釋放已持有的鎖
4. **注意活鎖問題**：加入隨機延遲避免無限重試
5. **搭配 std::unique_lock 使用更安全**：std::try_to_lock 標籤
6. **適用於有替代方案的場景**：不是用來替代 lock()

下一課預告

在 **課程 5.4：互斥鎖的常見錯誤** 中，我們將學習：

- 忘記解鎖的後果
- 重複鎖定的問題
- 在錯誤的執行緒解鎖
- 例外安全問題

這些都是實際開發中最常見的錯誤，學會避免它們非常重要！

準備好繼續嗎？

課程 5.4：互斥鎖的常見錯誤

第五階段：互斥鎖基礎 (std::mutex)

課程 5.4：互斥鎖的常見錯誤

引言

互斥鎖看似簡單，只有 `lock()` 和 `unlock()` 兩個主要操作，但實際開發中卻充滿陷阱。本課將系統性地分析最常見的錯誤，並學習如何避免它們。

一、錯誤總覽

互斥鎖常見錯誤 |

1. 忘記解鎖 (Forgetting to Unlock) |

→ 其他執行緒永遠等待 |

2. 重複鎖定 (Double Locking) |

→ 同一執行緒對同一 mutex 呼叫兩次 `lock()` |

3. 解鎖未鎖定的互斥鎖 (Unlocking Unowned Mutex) |

→ 未定義行為 |

4. 例外導致未解鎖 (Exception Without Unlock) |

→ 例外拋出後 `unlock()` 未執行 |

5. 鎖定順序不一致 (Inconsistent Lock Ordering) |

→ 導致死結 |

6. 保護範圍錯誤 (Incorrect Protection Scope) |

→ 臨界區段設計不當 |

二、錯誤 1：忘記解鎖

這是最常見也最危險的錯誤。

錯誤範例

```
// 檔案：lesson_5_4_forget_unlock.cpp
// 說明：忘記解鎖的錯誤示範

#include <iostream>
#include <thread>
#include <mutex>
#include <chrono>

std::mutex mtx;
int counter = 0;

void buggyIncrement() {
    mtx.lock();
    ++counter;
    std::cout << "Counter = " << counter << std::endl;
    // 🐛 忘記 unlock() !
    // mtx.unlock();
}

int main() {
    std::thread t1(buggyIncrement);
    t1.join();
    std::cout << "執行緒 1 完成" << std::endl;
    std::thread t2(buggyIncrement); // 🐛 永遠卡住！
    // 設定超時檢測
    std::this_thread::sleep_for(std::chrono::seconds(2));
    std::cout << "程式卡住了！（這行可能不會執行）" << std::endl;
    t2.join(); // 永遠不會返回
    return 0;
}
```

更隱蔽的情況：早期返回

```
// 檔案：lesson_5_4_early_return.cpp
// 說明：早期返回導致忘記解鎖
```

```

#include <iostream>
#include <mutex>

std::mutex mtx;

int getValue(int input) {
    mtx.lock();
    if (input < 0) {
        std::cout << "無效輸入" << std::endl;
        return -1; // 💀 提前返回，沒有 unlock！
    }
    if (input == 0) {
        std::cout << "零值" << std::endl;
        return 0; // 💀 又一個提前返回！
    }
    int result = input * 2;
    mtx.unlock();
    return result;
}

int main() {
    std::cout << getValue(10) << std::endl; // OK
    std::cout << getValue(-5) << std::endl; // 💀 鎖沒釋放
    std::cout << getValue(20) << std::endl; // 💀 永遠卡住
    return 0;
}

```

正確做法：使用 RAII

```

// 檔案：lesson_5_4_raii_fix.cpp
// 說明：使用 lock_guard 自動管理鎖

#include <iostream>
#include <mutex>

std::mutex mtx;

int getValueSafe(int input) {
    std::lock_guard<std::mutex> lock(mtx); // ✓ RAII

```

```

if (input < 0) {
    std::cout << "無效輸入" << std::endl;
    return -1; // ✓ lock_guard 解構時自動 unlock
}
if (input == 0) {
    std::cout << "零值" << std::endl;
    return 0; // ✓ 同樣會自動 unlock
}
return input * 2;
} // ✓ 函式結束，lock_guard 解構，自動 unlock

int main() {
    std::cout << getValueSafe(10) << std::endl;
    std::cout << getValueSafe(-5) << std::endl;
    std::cout << getValueSafe(20) << std::endl; // ✓ 正常執行
    return 0;
}

```

三、錯誤 2：重複鎖定

同一執行緒對同一個 `std::mutex` 呼叫兩次 `lock()`。

錯誤範例

```

// 檔案：lesson_5_4_double_lock.cpp
// 說明：重複鎖定的錯誤

#include <iostream>
#include <mutex>

std::mutex mtx;

void outerFunction();
void innerFunction();

void innerFunction() {
    mtx.lock(); // 💀 已經被 outerFunction 鎖定了！
    std::cout << "Inner function" << std::endl;
    mtx.unlock();
}

```

```

void outerFunction() {
    mtx.lock();
    std::cout << "Outer function" << std::endl;
    innerFunction(); // 💀 呼叫另一個也需要鎖的函式
    mtx.unlock();
}

int main() {
    outerFunction(); // 💀 死結！（或未定義行為）
    return 0;
}

```

重複鎖定的後果 |

std::mutex：未定義行為（通常是死結） |

outerFunction() |

| |

▼ |

mtx.lock() ← 成功 |

| |

▼ |

innerFunction() |

| |

▼ |

mtx.lock() ← 💀 同一執行緒再次嘗試鎖定 |

| 已經持有的鎖 |

▼ |

永遠等待自己釋放鎖（死結） |

| |

解決方案 1：使用 std::recursive_mutex

// 檔案：lesson_5_4_recursive_mutex.cpp

// 說明：使用遞迴互斥鎖解決重複鎖定

```
#include <iostream>
#include <mutex>

std::recursive_mutex rmtx; // ✓ 遞迴互斥鎖

void innerFunction() {
    rmtx.lock(); // ✓ 同一執行緒可以再次鎖定
    std::cout << "Inner function" << std::endl;
    rmtx.unlock();
}

void outerFunction() {
    rmtx.lock();
    std::cout << "Outer function" << std::endl;
    innerFunction(); // ✓ 正常運作
    rmtx.unlock();
}

int main() {
    outerFunction(); // ✓ 正常執行
    std::cout << "完成" << std::endl;
    return 0;
}
```

解決方案 2：重構程式碼（更好的做法）

```
// 檔案：lesson_5_4_refactor.cpp
// 說明：透過重構避免重複鎖定

#include <iostream>
#include <mutex>

std::mutex mtx;

// 內部實作（假設鎖已被持有，不加鎖）
void innerFunctionImpl() {
    std::cout << "Inner function" << std::endl;
}
```

```

// 公開介面（需要鎖）
void innerFunction() {
    std::lock_guard<std::mutex> lock(mtx);
    innerFunctionImpl();
}

void outerFunction() {
    std::lock_guard<std::mutex> lock(mtx);
    std::cout << "Outer function" << std::endl;
    innerFunctionImpl(); // ✓ 呼叫不加鎖的版本
}

int main() {
    outerFunction();
    std::cout << "完成" << std::endl;
    return 0;
}

```

四、錯誤 3：解鎖未鎖定的互斥鎖

// 檔案：lesson_5_4_unlock_unowned.cpp
 // 說明：解鎖未鎖定或不屬於自己的互斥鎖

```

#include <iostream>
#include <thread>
#include <mutex>

std::mutex mtx;

void wrongUnlock() {
    // 沒有 lock()
    mtx.unlock(); // 💀 未定義行為！
}

void wrongThread() {
    mtx.lock();
    std::thread t([&]() {

```

```

mtx.unlock(); // 💀 在不同執行緒解鎖！未定義行為！
});
t.join();
}

int main() {
// 這些都是錯誤的用法
// wrongUnlock();
// wrongThread();
std::cout << "這些函式都有問題，不要這樣做！" << std::endl;
return 0;
}

```

互斥鎖的所有權規則
規則 1：誰鎖定，誰解鎖
執行緒 A 呼叫 lock()，只有執行緒 A 能呼叫 unlock()
規則 2：先鎖定，後解鎖
必須先成功呼叫 lock() 或 try_lock()，才能呼叫 unlock()
規則 3：配對原則
每個成功的 lock()/try_lock() 必須有且只有一個 unlock()

五、錯誤 4：例外導致未解鎖

這是 C++ 特有的問題，也是 RAII 如此重要的原因。

錯誤範例

```

// 檔案：lesson_5_4_exception_unsafe.cpp
// 說明：例外導致 unlock 未執行

#include <iostream>
#include <mutex>
#include <stdexcept>

std::mutex mtx;

void riskyOperation(int value) {
    mtx.lock();
    std::cout << "開始處理 " << value << std::endl;
    if (value < 0) {
        throw std::invalid_argument("值不能為負數"); // 💀 例外拋出
        // unlock() 永遠不會執行！
    }
    std::cout << "處理完成" << std::endl;
    mtx.unlock();
}

int main() {
    try {
        riskyOperation(10); // OK
        riskyOperation(-5); // 💀 拋出例外，鎖沒釋放
    } catch (const std::exception& e) {
        std::cout << "捕獲例外：" << e.what() << std::endl;
    }
    // 此時 mtx 仍處於鎖定狀態！
    std::cout << "嘗試再次操作..." << std::endl;
    riskyOperation(20); // 💀 死結！
    return 0;
}

```

正確做法：使用 RAII

```

// 檔案：lesson_5_4_exception_safe.cpp
// 說明：使用 RAII 確保例外安全

#include <iostream>

```



```

#include <stdexcept>

std::mutex mtx;

void safeOperation(int value) {
    std::lock_guard<std::mutex> lock(mtx); // ✓ RAII
    std::cout << "開始處理 " << value << std::endl;
    if (value < 0) {
        throw std::invalid_argument("值不能為負數");
        // ✓ 例外拋出時，lock_guard 解構，自動 unlock
    }
    std::cout << "處理完成" << std::endl;
} // ✓ 正常返回時也會自動 unlock

int main() {
    try {
        safeOperation(10);
        safeOperation(-5); // 拋出例外
    } catch (const std::exception& e) {
        std::cout << "捕獲例外：" << e.what() << std::endl;
    }
    // ✓ 鎖已被正確釋放
    std::cout << "嘗試再次操作..." << std::endl;
    safeOperation(20); // ✓ 正常執行
    return 0;
}

```

輸出

```

開始處理 10
處理完成
開始處理 -5
捕獲例外：值不能為負數
嘗試再次操作...
開始處理 20
處理完成

```

六、錯誤 5：鎖定順序不一致

這會導致死結，我們在第八階段會詳細討論。

```
// 檔案：lesson_5_4_lock_order.cpp
// 說明：鎖定順序不一致導致死結

#include <iostream>
#include <thread>
#include <mutex>

std::mutex mutexA;
std::mutex mutexB;

void thread1() {
    mutexA.lock();
    std::cout << "Thread 1: locked A" << std::endl;
    std::this_thread::sleep_for(std::chrono::milliseconds(10));
    mutexB.lock(); // 💀 等待 Thread 2 釋放 B
    std::cout << "Thread 1: locked B" << std::endl;
    mutexB.unlock();
    mutexA.unlock();
}

void thread2() {
    mutexB.lock(); // ← 順序相反！
    std::cout << "Thread 2: locked B" << std::endl;
    std::this_thread::sleep_for(std::chrono::milliseconds(10));
    mutexA.lock(); // 💀 等待 Thread 1 釋放 A
    std::cout << "Thread 2: locked A" << std::endl;
    mutexA.unlock();
    mutexB.unlock();
}

int main() {
    std::thread t1(thread1);
    std::thread t2(thread2);
    t1.join();
    t2.join();
    // 💀 程式永遠不會到達這裡
    std::cout << "完成" << std::endl;
    return 0;
}
```

}

死結形成過程		
時間	Thread 1	Thread 2
T1	lock(A) ✓	
T2	lock(B) ✓	
T3	lock(B) 等待...	
T4	lock(A) 等待...	
	Thread 1 等待 B	Thread 2 等待 A
	B 被 Thread 2 持有 A 被 Thread 1 持有	
	💀 死結！	

解決方案：統一鎖定順序

// 總是先鎖 A，再鎖 B

```
void thread1() {  
    std::lock_guard<std::mutex> lockA(mutexA);  
    std::lock_guard<std::mutex> lockB(mutexB);  
    // 工作...  
}
```

```
void thread2() {  
    std::lock_guard<std::mutex> lockA(mutexA); // ✓ 同樣的順序  
    std::lock_guard<std::mutex> lockB(mutexB);  
    // 工作...  
}
```

七、錯誤 6：保護範圍錯誤

錯誤 6a：保護範圍太小

```
// 檔案：lesson_5_4_scope_too_small.cpp
// 說明：保護範圍太小，仍有競爭條件

#include <iostream>
#include <thread>
#include <mutex>
#include <vector>

std::mutex mtx;
std::vector<int> data;

void unsafeAppend(int value) {
    // 錯誤：檢查和操作分開保護
    mtx.lock();
    bool shouldAppend = (data.size() < 10);
    mtx.unlock();
    // 💡 此時其他執行緒可能已經改變了 data.size() !
    if (shouldAppend) {
        mtx.lock();
        data.push_back(value); // 💡 可能超過限制 !
        mtx.unlock();
    }
}

void safeAppend(int value) {
    // 正確：整個操作在同一個臨界區段
    std::lock_guard<std::mutex> lock(mtx);
    if (data.size() < 10) {
        data.push_back(value); // ✓ 安全
    }
}
```

錯誤 6b：保護範圍太大

```
// 檔案：lesson_5_4_scope_too_large.cpp
// 說明：保護範圍太大，降低效能
```

```

#include <iostream>
#include <thread>
#include <mutex>
#include <chrono>

std::mutex mtx;
int result = 0;

void inefficientWork(int input) {
    std::lock_guard<std::mutex> lock(mtx); // 🦋 整個函式都被鎖住
    // 這部分不需要鎖
    int temp = input * input;
    std::this_thread::sleep_for(std::chrono::milliseconds(100)); // 模擬耗時計算
    temp += input;
    // 只有這裡需要鎖
    result += temp;
}

void efficientWork(int input) {
    // 不需要鎖的部分
    int temp = input * input;
    std::this_thread::sleep_for(std::chrono::milliseconds(100));
    temp += input;
    // 只鎖必要的部分
    {
        std::lock_guard<std::mutex> lock(mtx);
        result += temp; // ✓ 最小化臨界區段
    }
}

int main() {
    auto start = std::chrono::steady_clock::now();
    std::thread t1(inefficientWork, 10);
    std::thread t2(inefficientWork, 20);
    t1.join();
    t2.join();
    auto mid = std::chrono::steady_clock::now();
    result = 0;
    std::thread t3(efficientWork, 10);
    std::thread t4(efficientWork, 20);

```

```

t4.join();
auto end = std::chrono::steady_clock::now();
auto inefficient_time = std::chrono::duration_cast<std::chrono::milliseconds>
(mid - start);
auto efficient_time = std::chrono::duration_cast<std::chrono::milliseconds>
(end - mid);
std::cout << "低效版本：" << inefficient_time.count() << " ms" << std::endl;
std::cout << "高效版本：" << efficient_time.count() << " ms" << std::endl;
return 0;
}

```

輸出

低效版本：200 ms （串行執行）

高效版本：100 ms （並行執行）

八、錯誤檢查清單

互斥鎖使用檢查清單
☐ 每個 <code>lock()</code> 都有對應的 <code>unlock()</code> ☐ 所有提前返回的路徑都會釋放鎖 → 使用 RAII (<code>lock_guard/unique_lock</code>) ☐ 例外拋出時鎖會被正確釋放 → 使用 RAII ☐ 沒有在同一執行緒重複鎖定 <code>std::mutex</code> → 如需要，使用 <code>recursive_mutex</code> 或重構 ☐ 只在持有鎖的執行緒呼叫 <code>unlock()</code> ☐ 多個互斥鎖的鎖定順序一致 → 或使用 <code>std::lock()</code> / <code>std::scoped_lock</code>

- | |
 - | □ 臨界區段盡可能小 |
 - | → 只包含必要的共享資料存取 |
 - | |
 - | □ 臨界區段內沒有阻塞操作 |
 - | → 避免 I/O、sleep、等待其他資源 |
 - | |
-

九、最佳實踐總結

// 檔案：lesson_5_4_best_practices.cpp

// 說明：互斥鎖最佳實踐範例

```
#include <iostream>
```

```
#include <mutex>
```

```
#include <vector>
```

```
#include <stdexcept>
```

```
class SafeCounter {
```

```
private:
```

```
mutable std::mutex mtx; // mutable 允許在 const 方法中鎖定
```

```
int count = 0;
```

```
public:
```

```
// ✓ 最佳實踐 1：使用 RAII
```

```
void increment() {
```

```
std::lock_guard<std::mutex> lock(mtx);
```

```
++count;
```

```
}
```

```
// ✓ 最佳實踐 2：最小化臨界區段
```

```
int getAndReset() {
```

```
int result;
```

```
{
```

```
std::lock_guard<std::mutex> lock(mtx);
```

```
result = count;
```

```
count = 0;
```

```
} // 鎖在這裡釋放
```

```
// 後續處理不需要鎖
```

```

return result;
}
// ✓ 最佳實踐 3: const 方法也要保護
int get() const {
    std::lock_guard<std::mutex> lock(mtx);
    return count;
}
// ✓ 最佳實踐 4: 例外安全
void riskyOperation(int value) {
    std::lock_guard<std::mutex> lock(mtx);
    if (value < 0) {
        throw std::invalid_argument("負值");
        // 例外拋出時 lock_guard 自動解鎖
    }
    count += value;
}
};

int main() {
    SafeCounter counter;
    counter.increment();
    counter.increment();
    std::cout << "Count: " << counter.get() << std::endl;
    try {
        counter.riskyOperation(-1);
    } catch (...) {
        std::cout << "例外被捕獲，鎖已正確釋放" << std::endl;
    }
    counter.increment(); // 正常運作
    std::cout << "Final count: " << counter.get() << std::endl;
    return 0;
}

```

十、本課重點回顧

錯誤類型	後果	解決方案
忘記解鎖	其他執行緒永遠等待	使用 RAII
重複鎖定	死結/未定義行為	recursive_mutex 或重構
解鎖未持有的鎖	未定義行為	遵守所有權規則
例外導致未解鎖	鎖洩漏	使用 RAII
鎖定順序不一致	死結	統一順序或 std::lock
保護範圍錯誤	競爭條件或效能差	仔細設計臨界區段

核心建議：永遠使用 RAII (lock_guard 或 unique_lock) 來管理鎖！

下一課預告

在 **課程 5.5：保護共享資料實作** 中，我們將：

- 實作一個完整的執行緒安全計數器類別
- 實作一個執行緒安全的銀行帳戶類別
- 學習如何設計執行緒安全的介面

準備好繼續嗎？

課程 5.5：保護共享資料實作

第五階段：互斥鎖基礎 (std::mutex)

課程 5.5：保護共享資料實作

引言

前幾課我們學習了 `std::mutex` 的基本操作和常見錯誤。現在，讓我們將這些知識付諸實踐，設計並實作完整的執行緒安全類別。這是多執行緒程式設計中最重要技能之一。

一、執行緒安全類別的設計原則

執行緒安全類別設計原則
1. 封裝原則
→ 互斥鎖和資料放在同一個類別中
→ 外部無法直接存取資料或鎖
2. 最小介面原則
→ 只暴露必要的操作
→ 避免返回內部資料的指標或引用
3. 原子操作原則
→ 每個公開方法應該是原子的
→ 檢查和操作應在同一個臨界區段
4. 不變量保護原則
→ 確保類別的不變量在任何時刻都成立
→ 臨界區段結束前恢復不變量

二、實作一：執行緒安全計數器

從最簡單的例子開始。

```
// 檔案：lesson_5_5_safe_counter.cpp
// 說明：執行緒安全的計數器類別

#include <iostream>
#include <thread>
#include <mutex>
#include <vector>

class ThreadSafeCounter {
private:
    mutable std::mutex mtx; // mutable：允許在 const 方法中使用
    int count = 0;
public:
    // 建構函式
```

```

ThreadSafeCounter() = default;
// 禁止複製（互斥鎖不可複製）
ThreadSafeCounter(const ThreadSafeCounter&) = delete;
ThreadSafeCounter& operator=(const ThreadSafeCounter&) = delete;
// 遞增
void increment() {
    std::lock_guard<std::mutex> lock(mtx);
    ++count;
}
// 遞減
void decrement() {
    std::lock_guard<std::mutex> lock(mtx);
    --count;
}
// 增加指定值
void add(int value) {
    std::lock_guard<std::mutex> lock(mtx);
    count += value;
}
// 獲取當前值
int get() const {
    std::lock_guard<std::mutex> lock(mtx);
    return count;
}
// 重設並返回舊值（原子操作）
int reset() {
    std::lock_guard<std::mutex> lock(mtx);
    int oldValue = count;
    count = 0;
    return oldValue;
}
// 條件遞增：只有當前值小於 max 時才遞增
bool incrementIfLessThan(int max) {
    std::lock_guard<std::mutex> lock(mtx);
    if (count < max) {
        ++count;
        return true;
    }
    return false;
}

```

```
// 測試
void workerIncrement(ThreadSafeCounter& counter, int times) {
    for (int i = 0; i < times; ++i) {
        counter.increment();
    }
}

int main() {
    ThreadSafeCounter counter;
    const int numThreads = 4;
    const int incrementsPerThread = 100000;
    std::vector<std::thread> threads;
    for (int i = 0; i < numThreads; ++i) {
        threads.emplace_back(workerIncrement, std::ref(counter), incrementsPerThread);
    }
    for (auto& t : threads) {
        t.join();
    }
    int expected = numThreads * incrementsPerThread;
    int actual = counter.get();
    std::cout << "預期值：" << expected << std::endl;
    std::cout << "實際值：" << actual << std::endl;
    std::cout << "結果：" << (expected == actual ? "✓ 正確" : "✗ 錯誤") <<
    std::endl;
    return 0;
}
```

輸出

預期值：400000

實際值：400000

結果：✓ 正確

三、為什麼禁止複製？

| 為什麼含有 `mutex` 的類別應禁止複製？ |

| |

| 問題 1：`std::mutex` 本身不可複製 |

| `std::mutex mtx1;` |

| `std::mutex mtx2 = mtx1; // 編譯錯誤！` |

| 問題 2：複製語意不明確 |

| 如果允許複製： |

- 複製時原物件被鎖定怎麼辦？ |
- 新物件應該共享鎖還是有自己的鎖？ |
- 複製後兩個物件是否獨立？ |

| 解決方案 |

| 1. 禁止複製和賦值 |

| `Counter(const Counter&) = delete;` |

| `Counter& operator=(const Counter&) = delete;` |

| 2. 如果需要複製語意，明確定義行為 |

| (見下方範例) |

如果確實需要複製

// 檔案：lesson_5_5_copyable_counter.cpp

// 說明：可複製的執行緒安全計數器（明確定義複製行為）

```
#include <iostream>
```

```
#include <mutex>
```

```
class CopyableCounter {
```

```
private:
```

```
    mutable std::mutex mtx;
```

```
    int count = 0;
```

```
public:
```

```
    CopyableCounter() = default;
```

```

// 明確定義複製建構函式
CopyableCounter(const CopyableCounter& other) {
    std::lock_guard<std::mutex> lock(other.mtx); // 鎖定來源
    count = other.count; // 複製值
    // 新物件有自己的 mutex (預設建構)
}
// 明確定義複製賦值運算子
CopyableCounter& operator=(const CopyableCounter& other) {
    if (this != &other) {
        // 需要同時鎖定兩個物件，注意死結！
        std::scoped_lock lock(mtx, other.mtx);
        count = other.count;
    }
    return *this;
}
void increment() {
    std::lock_guard<std::mutex> lock(mtx);
    ++count;
}
int get() const {
    std::lock_guard<std::mutex> lock(mtx);
    return count;
}
};

int main() {
    CopyableCounter c1;
    c1.increment();
    c1.increment();
    CopyableCounter c2 = c1; // 複製
    std::cout << "c1 = " << c1.get() << std::endl; // 2
    std::cout << "c2 = " << c2.get() << std::endl; // 2
    c1.increment();
    std::cout << "c1 = " << c1.get() << std::endl; // 3
    std::cout << "c2 = " << c2.get() << std::endl; // 2 (獨立)
    return 0;
}

```

四、實作二：執行緒安全銀行帳戶

更複雜的實際案例。

```
// 檔案：lesson_5_5_bank_account.cpp
// 說明：執行緒安全的銀行帳戶類別

#include <iostream>
#include <thread>
#include <mutex>
#include <vector>
#include <string>
#include <stdexcept>

class BankAccount {
private:
    mutable std::mutex mtx;
    std::string accountId;
    double balance;
public:
    BankAccount(const std::string& id, double initialBalance)
        : accountId(id), balance(initialBalance) {
        if (initialBalance < 0) {
            throw std::invalid_argument("初始餘額不能為負");
        }
    }
    // 禁止複製
    BankAccount(const BankAccount&) = delete;
    BankAccount& operator=(const BankAccount&) = delete;
    // 允許移動
    BankAccount(BankAccount&& other) noexcept {
        std::lock_guard<std::mutex> lock(other.mtx);
        accountId = std::move(other.accountId);
        balance = other.balance;
        other.balance = 0;
    }
    // 存款
    void deposit(double amount) {
        if (amount <= 0) {
            throw std::invalid_argument("存款金額必須為正");
        }
        std::lock_guard<std::mutex> lock(mtx);
```

```

balance += amount;
std::cout << "[" << accountId << "]" 存款 " << amount
<< ", 餘額:" << balance << std::endl;
}
// 提款
bool withdraw(double amount) {
    if (amount <= 0) {
        throw std::invalid_argument("提款金額必須為正");
    }
    std::lock_guard<std::mutex> lock(mtx);
    if (balance >= amount) {
        balance -= amount;
        std::cout << "[" << accountId << "]" 提款 " << amount
        << ", 餘額:" << balance << std::endl;
        return true;
    }
    std::cout << "[" << accountId << "]" 提款失敗：餘額不足" << std::endl;
    return false;
}
// 查詢餘額
double getBalance() const {
    std::lock_guard<std::mutex> lock(mtx);
    return balance;
}
// 獲取帳戶 ID
std::string getId() const {
    std::lock_guard<std::mutex> lock(mtx);
    return accountId;
}
// 轉帳（靜態方法，需要鎖定兩個帳戶）
static bool transfer(BankAccount& from, BankAccount& to, double amount) {
    if (&from == &to) {
        return false; // 不能轉給自己
    }
    if (amount <= 0) {
        throw std::invalid_argument("轉帳金額必須為正");
    }
    // 使用 std::scoped_lock 同時鎖定兩個帳戶，避免死結
    std::scoped_lock lock(from.mtx, to.mtx);
    if (from.balance >= amount) {

```



```

to.balance += amount;
std::cout << "[轉帳] " << from.accountId << " → " << to.accountId
<< ":" << amount << std::endl;
return true;
}
std::cout << "[轉帳失敗] " << from.accountId << " 餘額不足" << std::endl;
return false;
}
};

```

// 模擬交易

```

void simulateTransactions(BankAccount& account, int id) {
for (int i = 0; i < 5; ++i) {
account.deposit(100);
account.withdraw(50);
}
}

```

```

int main() {
BankAccount alice("Alice", 1000);
BankAccount bob("Bob", 500);
std::cout << "=== 初始狀態 ===" << std::endl;
std::cout << "Alice: " << alice.getBalance() << std::endl;
std::cout << "Bob: " << bob.getBalance() << std::endl;
std::cout << "\n=== 並行交易 ===" << std::endl;
std::thread t1([&]() {
for (int i = 0; i < 3; ++i) {
BankAccount::transfer(alice, bob, 100);
}
});
std::thread t2([&]() {
for (int i = 0; i < 3; ++i) {
BankAccount::transfer(bob, alice, 50);
}
});
std::thread t3(simulateTransactions, std::ref(alice), 1);
std::thread t4(simulateTransactions, std::ref(bob), 2);
t1.join();
t2.join();
t3.join();
t4.join();
}

```

```

std::cout << "\n=== 最終狀態 ===" << std::endl;
std::cout << "Alice: " << alice.getBalance() << std::endl;
std::cout << "Bob: " << bob.getBalance() << std::endl;
std::cout << "總額: " << (alice.getBalance() + bob.getBalance()) << std::endl;
std::cout << "（初始總額為 1500，應保持不變）" << std::endl;
return 0;
}

```

五、介面設計的陷阱

危險：返回內部資料的引用

// 檔案：lesson_5_5_dangerous_interface.cpp
 // 說明：危險的介面設計

```

#include <iostream>
#include <mutex>
#include <vector>

class DangerousContainer {
private:
    std::mutex mtx;
    std::vector<int> data;
public:
    // 💀 危險！返回內部資料的引用
    std::vector<int>& getData() {
        std::lock_guard<std::mutex> lock(mtx);
        return data; // 鎖在這裡就釋放了！
    }
    // 💀 危險！返回內部資料的指標
    int* getFirst() {
        std::lock_guard<std::mutex> lock(mtx);
        if (!data.empty()) {
            return &data[0]; // 返回後鎖就釋放了！
        }
        return nullptr;
    }
    void add(int value) {
        std::lock_guard<std::mutex> lock(mtx);
    }
}

```

```

data.push_back(value);
}
};

/*
* 問題分析：
*
* Thread A: Thread B:
* auto& vec = container.getData();
* // 鎖已釋放！ container.add(100);
* vec.push_back(1); // 同時修改！
* // 🦋 競爭條件！
*/

```

安全：複製資料或提供安全的操作

// 檔案：lesson_5_5_safe_interface.cpp
// 說明：安全的介面設計

```

#include <iostream>
#include <mutex>
#include <vector>
#include <functional>
#include <algorithm>

class SafeContainer {
private:
mutable std::mutex mtx;
std::vector<int> data;
public:
// ✓ 安全：返回複本
std::vector<int> getData() const {
std::lock_guard<std::mutex> lock(mtx);
return data; // 返回複本，不是引用
}
// ✓ 安全：返回值的複本
int getAt(size_t index) const {
std::lock_guard<std::mutex> lock(mtx);
if (index < data.size()) {
return data[index]; // 返回值，不是引用
}
}
}

```

```

}
throw std::out_of_range("索引超出範圍");
}
// ✓ 安全：提供操作而非資料存取
void add(int value) {
    std::lock_guard<std::mutex> lock(mtx);
    data.push_back(value);
}
// ✓ 安全：使用回呼函式處理資料
void forEach(const std::function<void(int)>& func) const {
    std::lock_guard<std::mutex> lock(mtx);
    for (int value : data) {
        func(value);
    }
}
// ✓ 安全：在鎖保護下執行操作
template<typename Func>
auto withLock(Func&& func) -> decltype(func(data)) {
    std::lock_guard<std::mutex> lock(mtx);
    return func(data);
}
// ✓ 安全：批次操作
void addAll(const std::vector<int>& values) {
    std::lock_guard<std::mutex> lock(mtx);
    data.insert(data.end(), values.begin(), values.end());
}
size_t size() const {
    std::lock_guard<std::mutex> lock(mtx);
    return data.size();
}
bool empty() const {
    std::lock_guard<std::mutex> lock(mtx);
    return data.empty();
}
};

int main() {
    SafeContainer container;
    container.add(10);
    container.add(20);
    container.add(30);
}

```

```

// 使用 forEach
std::cout << "forEach: ";
container.forEach([](int v) {
std::cout << v << " ";
});
std::cout << std::endl;
// 使用 withLock 進行複雜操作
int sum = container.withLock([](std::vector<int>& data) {
int total = 0;
for (int v : data) {
total += v;
}
return total;
});
std::cout << "Sum: " << sum << std::endl;
// 獲取複本
auto copy = container.getData();
std::cout << "Copy size: " << copy.size() << std::endl;
return 0;
}

```

六、實作三：執行緒安全的 Logger

實際應用中常見的日誌系統。

```

// 檔案：lesson_5_5_thread_safe_logger.cpp
// 說明：執行緒安全的日誌記錄器

```

```

#include <iostream>
#include <thread>
#include <mutex>
#include <string>
#include <sstream>
#include <chrono>
#include <iomanip>
#include <vector>
#include <fstream>

class ThreadSafeLogger {

```

```

public:
enum class Level {
DEBUG,
INFO,
WARNING,
ERROR
};
private:
mutable std::mutex mtx;
std::ostream& output;
Level minLevel;
bool includeTimestamp;
bool includeThreadId;
std::string levelToString(Level level) const {
switch (level) {
case Level::DEBUG: return "DEBUG";
case Level::INFO: return "INFO ";
case Level::WARNING: return "WARN ";
case Level::ERROR: return "ERROR";
default: return "?????";
}
}
std::string getTimestamp() const {
auto now = std::chrono::system_clock::now();
auto time = std::chrono::system_clock::to_time_t(now);
auto ms = std::chrono::duration_cast<std::chrono::milliseconds>(
now.time_since_epoch()) % 1000;
std::ostringstream oss;
oss << std::put_time(std::localtime(&time), "%H:%M:%S");
oss << '.' << std::setfill('0') << std::setw(3) << ms.count();
return oss.str();
}
public:
ThreadSafeLogger(std::ostream& out = std::cout,
Level level = Level::DEBUG,
bool timestamp = true,
bool threadId = true)
: output(out)
, minLevel(level)
, includeTimestamp(timestamp)
, includeThreadId(threadId) {}

```

```

// 禁止複製
ThreadSafeLogger(const ThreadSafeLogger&) = delete;
ThreadSafeLogger& operator=(const ThreadSafeLogger&) = delete;
void setLevel(Level level) {
    std::lock_guard<std::mutex> lock(mtx);
    minLevel = level;
}
void log(Level level, const std::string& message) {
    if (level < minLevel) {
        return; // 過濾低級別日誌
    }
    // 在鎖外準備訊息（減少臨界區段）
    std::ostringstream oss;
    if (includeTimestamp) {
        oss << "[" << getTimestamp() << "] ";
    }
    oss << "[" << levelToString(level) << "] ";
    if (includeThreadId) {
        oss << "[T:" << std::this_thread::get_id() << "] ";
    }
    oss << message << std::endl;
    std::string formatted = oss.str();
    // 只在輸出時鎖定
    std::lock_guard<std::mutex> lock(mtx);
    output << formatted;
    output.flush();
}
void debug(const std::string& msg) { log(Level::DEBUG, msg); }
void info(const std::string& msg) { log(Level::INFO, msg); }
void warning(const std::string& msg) { log(Level::WARNING, msg); }
void error(const std::string& msg) { log(Level::ERROR, msg); }
};

// 全域 Logger（實際專案中可能使用單例模式）
ThreadSafeLogger logger;

void worker(int id) {
    logger.info("Worker " + std::to_string(id) + " 開始");
    for (int i = 0; i < 3; ++i) {
        logger.debug("Worker " + std::to_string(id) +

```

```

std::this_thread::sleep_for(std::chrono::milliseconds(10));
}
logger.info("Worker " + std::to_string(id) + " 結束");
}

int main() {
logger.info("=== 程式開始 ===");
std::vector<std::thread> threads;
for (int i = 1; i <= 3; ++i) {
threads.emplace_back(worker, i);
}
for (auto& t : threads) {
t.join();
}
logger.info("=== 程式結束 ===");
return 0;
}

```

輸出範例

```

[14:32:15.123] [INFO ] [T:140001] === 程式開始 ===
[14:32:15.124] [INFO ] [T:140002] Worker 1 開始
[14:32:15.124] [INFO ] [T:140003] Worker 2 開始
[14:32:15.124] [INFO ] [T:140004] Worker 3 開始
[14:32:15.124] [DEBUG] [T:140002] Worker 1 執行任務 0
[14:32:15.124] [DEBUG] [T:140003] Worker 2 執行任務 0
[14:32:15.125] [DEBUG] [T:140004] Worker 3 執行任務 0
...
[14:32:15.156] [INFO ] [T:140002] Worker 1 結束
[14:32:15.157] [INFO ] [T:140003] Worker 2 結束
[14:32:15.157] [INFO ] [T:140004] Worker 3 結束
[14:32:15.158] [INFO ] [T:140001] === 程式結束 ===

```

七、實作四：執行緒安全的設定管理器

```

// 檔案：lesson_5_5_config_manager.cpp
// 說明：執行緒安全的設定管理器

```



```

#include <iostream>
#include <thread>
#include <mutex>
#include <map>
#include <string>
#include <optional>
#include <vector>

class ConfigManager {
private:
    mutable std::mutex mtx;
    std::map<std::string, std::string> config;
public:
    // 設定値
    void set(const std::string& key, const std::string& value) {
        std::lock_guard<std::mutex> lock(mtx);
        config[key] = value;
    }
    // 獲取值 (使用 optional 表示可能不存在)
    std::optional<std::string> get(const std::string& key) const {
        std::lock_guard<std::mutex> lock(mtx);
        auto it = config.find(key);
        if (it != config.end()) {
            return it->second;
        }
        return std::nullopt;
    }
    // 獲取值，若不存在則返回預設值
    std::string getOrDefault(const std::string& key,
        const std::string& defaultValue) const {
        std::lock_guard<std::mutex> lock(mtx);
        auto it = config.find(key);
        if (it != config.end()) {
            return it->second;
        }
        return defaultValue;
    }
    // 檢查是否存在
    bool has(const std::string& key) const {

```

```

return config.find(key) != config.end();
}
// 刪除
bool remove(const std::string& key) {
    std::lock_guard<std::mutex> lock(mtx);
    return config.erase(key) > 0;
}
// 獲取所有 key
std::vector<std::string> keys() const {
    std::lock_guard<std::mutex> lock(mtx);
    std::vector<std::string> result;
    result.reserve(config.size());
    for (const auto& [key, value] : config) {
        result.push_back(key);
    }
    return result;
}
// 批次設定
void setAll(const std::map<std::string, std::string>& values) {
    std::lock_guard<std::mutex> lock(mtx);
    for (const auto& [key, value] : values) {
        config[key] = value;
    }
}
// 原子性的「若不存在則設定」
bool setIfAbsent(const std::string& key, const std::string& value) {
    std::lock_guard<std::mutex> lock(mtx);
    if (config.find(key) == config.end()) {
        config[key] = value;
        return true;
    }
    return false;
}
// 原子性的「比較並交換」
bool compareAndSet(const std::string& key,
    const std::string& expected,
    const std::string& newValue) {
    std::lock_guard<std::mutex> lock(mtx);
    auto it = config.find(key);
    if (it != config.end() && it->second == expected) {

```

```

return true;
}
return false;
}
size_t size() const {
    std::lock_guard<std::mutex> lock(mtx);
    return config.size();
}
};

// 測試
void configWriter(ConfigManager& cfg, int id) {
    for (int i = 0; i < 5; ++i) {
        std::string key = "key_" + std::to_string(i);
        std::string value = "writer" + std::to_string(id) + "_value" +
            std::to_string(i);
        cfg.set(key, value);
        std::this_thread::sleep_for(std::chrono::milliseconds(1));
    }
}

void configReader(ConfigManager& cfg, int id) {
    for (int i = 0; i < 5; ++i) {
        std::string key = "key_" + std::to_string(i);
        auto value = cfg.get(key);
        if (value) {
            std::cout << "Reader " << id << ": " << key << " = " << *value << std::endl;
        }
        std::this_thread::sleep_for(std::chrono::milliseconds(2));
    }
}

int main() {
    ConfigManager config;
    // 初始設定
    config.set("app.name", "MyApp");
    config.set("app.version", "1.0.0");
    std::vector<std::thread> threads;
    // 啟動寫入者
    threads.emplace_back(configWriter, std::ref(config), 1);
    threads.emplace_back(configWriter, std::ref(config), 2);

```

```

// 啟動讀取者
threads.emplace_back(configReader, std::ref(config), 1);
threads.emplace_back(configReader, std::ref(config), 2);
for (auto& t : threads) {
t.join();
}
std::cout << "\n=== 最終設定 ===" << std::endl;
for (const auto& key : config.keys()) {
std::cout << key << " = " << config.getDefault(key, "") << std::endl;
}
return 0;
}

```

八、設計模式：將鎖封裝在類別中

執行緒安全類別設計總結

```

| |
| class ThreadSafeClass { |
| private: |
| mutable std::mutex mtx; // 1. 鎖是私有的 |
| Data data; // 2. 資料是私有的 |
| |
| public: |
| // 3. 禁止複製 |
| ThreadSafeClass(const ThreadSafeClass&) = delete; |
| ThreadSafeClass& operator=(const ThreadSafeClass&) = delete; |
| |
| // 4. 所有公開方法都是原子的 |
| void operation() { |
| std::lock_guard<std::mutex> lock(mtx); |
| // 操作 data |
| } |
| |
| // 5. 不返回內部資料的引用/指標 |
| Data getData() const { |
| std::lock_guard<std::mutex> lock(mtx); |

```

```
| return data; // 返回複本 |  
| } |  
| }; |  
| |
```

九、本課重點回顧

1. **封裝原則**：鎖和資料放在同一類別，外部無法直接存取
2. **禁止複製**：含有 mutex 的類別通常應禁止複製
3. **mutable mutex**：允許在 const 方法中使用鎖
4. **安全介面**：不返回內部資料的引用或指標
5. **原子操作**：檢查和操作應在同一臨界區段
6. **減少臨界區段**：只鎖定必要的部分
7. **使用 scoped_lock**：同時鎖定多個互斥鎖時避免死結

下一課預告

在 **課程 5.6：互斥鎖的效能考量** 中，我們將學習：

- 鎖的開銷測量
- 粗粒度鎖 vs 細粒度鎖
- 鎖競爭的影響
- 效能優化策略

這是第五階段的最後一課，將為進入 RAII 鎖管理器做好準備！

準備好繼續嗎？

課程 5.6：互斥鎖的效能考量

第五階段：互斥鎖基礎 (std::mutex)

課程 5.6：互斥鎖的效能考量

引言

互斥鎖是保護共享資料的利器，但它不是免費的。濫用鎖會導致程式效能急劇下降，甚至比單執行緒還慢。本課將深入探討鎖的效能特性，並學習如何在正確性與效能之間取得平衡。

一、鎖的開銷來源

鎖的開銷來源

1. 原子操作開銷

- CAS、記憶體屏障等指令比普通指令慢
- 約 10-50 個 CPU 週期

2. 快取一致性開銷

- 鎖變數在多核心間同步
- 快取行失效與重新載入
- 約 100-500 個 CPU 週期

3. 競爭開銷

- 自旋等待消耗 CPU
- 執行緒睡眠與喚醒（系統呼叫）
- 上下文切換：1000-10000+ 個 CPU 週期

4. 序列化開銷

- 臨界區段只能串行執行
- 並行度降低

二、測量鎖的基本開銷

```
// 檔案：lesson_5_6_lock_overhead.cpp  
// 說明：測量互斥鎖的基本開銷
```

```
#include <iostream>  
#include <thread>  
#include <mutex>  
#include <chrono>  
#include <vector>  
#include <iomanip>
```

```

std::mutex mtx;

// 無鎖操作
void noLockOperation(long long& counter, int iterations) {
    for (int i = 0; i < iterations; ++i) {
        ++counter;
    }
}

// 有鎖操作
void withLockOperation(long long& counter, int iterations) {
    for (int i = 0; i < iterations; ++i) {
        std::lock_guard<std::mutex> lock(mtx);
        ++counter;
    }
}

// 計時函式
template<typename Func>
double measureTime(Func&& func) {
    auto start = std::chrono::high_resolution_clock::now();
    func();
    auto end = std::chrono::high_resolution_clock::now();
    return std::chrono::duration<double, std::nano>(end - start).count();
}

int main() {
    const int iterations = 1000000;
    long long counter = 0;
    std::cout << "=== 互斥鎖開銷測量 ===" << std::endl;
    std::cout << "迭代次數：" << iterations << std::endl << std::endl;
    // 測量無鎖操作
    counter = 0;
    double noLockTime = measureTime([&]() {
        noLockOperation(counter, iterations);
    });
    // 測量有鎖操作（單執行緒，無競爭）
    counter = 0;
    double withLockTime = measureTime([&]() {

```

```

});
double lockOverhead = (withLockTime - noLockTime) / iterations;
std::cout << std::fixed << std::setprecision(2);
std::cout << "無鎖操作總時間：" << noLockTime / 1e6 << " ms" << std::endl;
std::cout << "有鎖操作總時間：" << withLockTime / 1e6 << " ms" << std::endl;
std::cout << "每次鎖操作開銷：" << lockOverhead << " ns" << std::endl;
std::cout << "效能比率：" << withLockTime / noLockTime << "x 慢" << std::endl;
return 0;
}

```

典型輸出

```

=== 互斥鎖開銷測量 ===
迭代次數：1000000

```

```

無鎖操作總時間：2.15 ms
有鎖操作總時間：45.32 ms
每次鎖操作開銷：43.17 ns
效能比率：21.08x 慢

```

三、競爭對效能的影響

```

// 檔案：lesson_5_6_contention.cpp
// 說明：測量鎖競爭對效能的影響

#include <iostream>
#include <thread>
#include <mutex>
#include <chrono>
#include <vector>
#include <iomanip>

std::mutex mtx;
long long sharedCounter = 0;

void incrementWithLock(int iterations) {
    for (int i = 0; i < iterations; ++i) {

```



```

std::lock_guard<std::mutex> lock(mtx);
++sharedCounter;
}
}

double runTest(int numThreads, int totalIterations) {
    sharedCounter = 0;
    int iterationsPerThread = totalIterations / numThreads;
    std::vector<std::thread> threads;
    auto start = std::chrono::high_resolution_clock::now();
    for (int i = 0; i < numThreads; ++i) {
        threads.emplace_back(incrementWithLock, iterationsPerThread);
    }
    for (auto& t : threads) {
        t.join();
    }
    auto end = std::chrono::high_resolution_clock::now();
    return std::chrono::duration<double, std::milli>(end - start).count();
}

int main() {
    const int totalIterations = 1000000;
    std::cout << "=== 鎖競爭效能測試 ===" << std::endl;
    std::cout << "總迭代次數：" << totalIterations << std::endl << std::endl;
    std::cout << std::setw(10) << "執行緒數"
    << std::setw(15) << "時間 (ms)"
    << std::setw(15) << "相對效能"
    << std::endl;
    std::cout << std::string(40, '-') << std::endl;
    double baseTime = runTest(1, totalIterations);
    std::cout << std::fixed << std::setprecision(2);
    std::cout << std::setw(10) << 1
    << std::setw(15) << baseTime
    << std::setw(15) << "1.00x"
    << std::endl;
    for (int numThreads : {2, 4, 8, 16}) {
        double time = runTest(numThreads, totalIterations);
        double ratio = time / baseTime;
        std::cout << std::setw(10) << numThreads
        << std::setw(15) << time

```

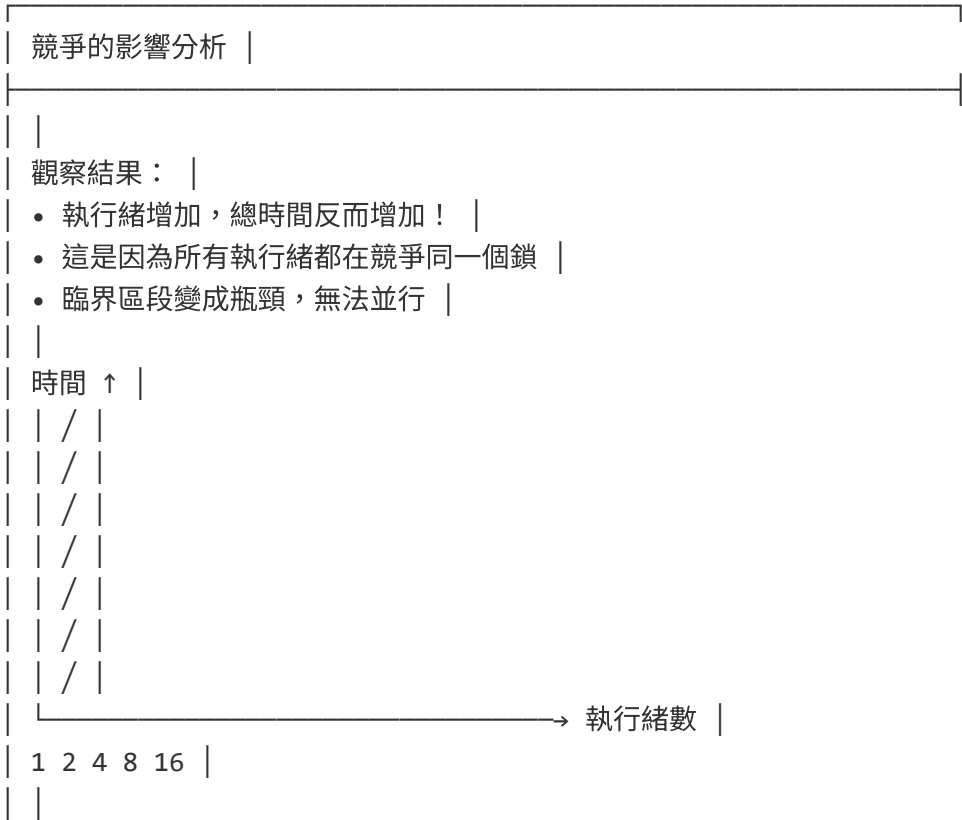
```
<< std::endl;
}
return 0;
}
```

典型輸出

=== 鎖競爭效能測試 ===
總迭代次數：1000000

執行緒數 時間 (ms) 相對效能

1	42.35	1.00x
2	98.67	2.33x
4	187.42	4.43x
8	412.56	9.74x
16	856.23	20.22x



結論：高競爭場景下，多執行緒比單執行緒還慢！

四、粗粒度鎖 vs 細粒度鎖

粗粒度鎖 vs 細粒度鎖

粗粒度鎖 (Coarse-grained Locking)

- 一個鎖保護大量資料
- 實作簡單，不易出錯
- 並行度低，競爭嚴重

一個大鎖

資料1 | 資料2 | 資料3 | 資料4

細粒度鎖 (Fine-grained Locking)

- 多個鎖保護不同資料
- 實作複雜，容易死結
- 並行度高，競爭減少

鎖1 | 鎖2 | 鎖3 | 鎖4

資料1 | 資料2 | 資料3 | 資料4

比較實作

```
// 檔案：lesson_5_6_lock_granularity.cpp
// 說明：比較粗粒度鎖與細粒度鎖的效能

#include <iostream>
#include <thread>
#include <mutex>
#include <chrono>
#include <vector>
#include <array>
#include <iomanip>

const int NUM_BUCKETS = 16;
const int ITERATIONS = 100000;

// 粗粒度鎖：一個鎖保護所有資料
class CoarseGrainedCounters {
private:
    std::mutex mtx;
    std::array<long long, NUM_BUCKETS> counters{};
public:
    void increment(int bucket) {
        std::lock_guard<std::mutex> lock(mtx); // 鎖整個結構
        ++counters[bucket % NUM_BUCKETS];
    }
    long long get(int bucket) const {
        return counters[bucket % NUM_BUCKETS];
    }
    long long total() const {
        long long sum = 0;
        for (auto c : counters) sum += c;
        return sum;
    }
};

// 細粒度鎖：每個桶一個鎖
class FineGrainedCounters {
private:
```

```

std::array<std::mutex, NUM_BUCKETS> mutexes;
std::array<long long, NUM_BUCKETS> counters{};
public:
void increment(int bucket) {
int idx = bucket % NUM_BUCKETS;
std::lock_guard<std::mutex> lock(mutexes[idx]); // 只鎖一個桶
++counters[idx];
}
long long get(int bucket) const {
return counters[bucket % NUM_BUCKETS];
}
long long total() const {
long long sum = 0;
for (auto c : counters) sum += c;
return sum;
}
};

template<typename Counter>
void worker(Counter& counter, int threadId, int iterations) {
for (int i = 0; i < iterations; ++i) {
// 每個執行緒存取不同的桶 (模擬真實場景)
counter.increment(threadId * 1000 + i);
}
}

template<typename Counter>
double runTest(int numThreads) {
Counter counter;
std::vector<std::thread> threads;
auto start = std::chrono::high_resolution_clock::now();
for (int i = 0; i < numThreads; ++i) {
threads.emplace_back(worker<Counter>, std::ref(counter), i, ITERATIONS);
}
for (auto& t : threads) {
t.join();
}
auto end = std::chrono::high_resolution_clock::now();
return std::chrono::duration<double, std::milli>(end - start).count();
}

```

```

int main() {
    std::cout << "=== 鎖粒度效能比較 ===" << std::endl;
    std::cout << "桶數量：" << NUM_BUCKETS << std::endl;
    std::cout << "每執行緒迭代：" << ITERATIONS << std::endl << std::endl;
    std::cout << std::setw(10) << "執行緒數"
    << std::setw(18) << "粗粒度 (ms)"
    << std::setw(18) << "細粒度 (ms)"
    << std::setw(12) << "加速比"
    << std::endl;
    std::cout << std::string(58, '-') << std::endl;
    std::cout << std::fixed << std::setprecision(2);
    for (int numThreads : {1, 2, 4, 8, 16}) {
        double coarseTime = runTest<CoarseGrainedCounters>(numThreads);
        double fineTime = runTest<FineGrainedCounters>(numThreads);
        double speedup = coarseTime / fineTime;
        std::cout << std::setw(10) << numThreads
        << std::setw(18) << coarseTime
        << std::setw(18) << fineTime
        << std::setw(11) << speedup << "x"
        << std::endl;
    }
    return 0;
}

```

典型輸出

=== 鎖粒度效能比較 ===

桶數量：16

每執行緒迭代：100000

執行緒數 粗粒度 (ms) 細粒度 (ms) 加速比

```

-----
1 4.52 4.78 0.95x
2 12.34 5.21 2.37x
4 28.67 6.45 4.45x
8 65.43 8.92 7.34x
16 142.56 12.34 11.55x

```

五、減少臨界區段大小

// 檔案：lesson_5_6_minimize_critical_section.cpp

// 說明：減少臨界區段大小提升效能

```
#include <iostream>
#include <thread>
#include <mutex>
#include <chrono>
#include <vector>
#include <cmath>
#include <iomanip>
```

```
std::mutex mtx;
double result = 0;
```

// 耗時計算（模擬複雜運算）

```
double expensiveComputation(int input) {
    double sum = 0;
    for (int i = 0; i < 1000; ++i) {
        sum += std::sin(input + i) * std::cos(input - i);
    }
    return sum;
}
```

// 差的做法：在鎖內進行計算

```
void badApproach(int start, int count) {
    for (int i = start; i < start + count; ++i) {
        std::lock_guard<std::mutex> lock(mtx); // 鎖定
        double computed = expensiveComputation(i); // 💀 在鎖內計算！
        result += computed;
    } // 解鎖
}
```

// 好的做法：在鎖外進行計算

```
void goodApproach(int start, int count) {
    for (int i = start; i < start + count; ++i) {
        double computed = expensiveComputation(i); // ✓ 在鎖外計算
        std::lock_guard<std::mutex> lock(mtx); // 鎖定
        result += computed; // 只鎖定必要的部分
    } // 解鎖
}
```

```
}
```

```
// 更好的做法：批次累加
```

```
void betterApproach(int start, int count) {  
    double localSum = 0; // 本地累加  
    for (int i = start; i < start + count; ++i) {  
        localSum += expensiveComputation(i); // 完全不需要鎖  
    }  
    std::lock_guard<std::mutex> lock(mtx); // 只鎖定一次  
    result += localSum;  
}
```

```
template<typename Func>  
double runTest(Func&& func, int numThreads, int totalWork) {  
    result = 0;  
    int workPerThread = totalWork / numThreads;  
    std::vector<std::thread> threads;  
    auto start = std::chrono::high_resolution_clock::now();  
    for (int i = 0; i < numThreads; ++i) {  
        threads.emplace_back(func, i * workPerThread, workPerThread);  
    }  
    for (auto& t : threads) {  
        t.join();  
    }  
    auto end = std::chrono::high_resolution_clock::now();  
    return std::chrono::duration<double, std::milli>(end - start).count();  
}
```

```
int main() {  
    const int totalWork = 1000;  
    const int numThreads = 4;  
    std::cout << "=== 臨界區段大小對效能的影響 ===" << std::endl;  
    std::cout << "執行緒數：" << numThreads << std::endl;  
    std::cout << "總工作量：" << totalWork << std::endl << std::endl;  
    std::cout << std::fixed << std::setprecision(2);  
    double badTime = runTest(badApproach, numThreads, totalWork);  
    std::cout << "差的做法（鎖內計算）：" << badTime << " ms" << std::endl;  
    double goodTime = runTest(goodApproach, numThreads, totalWork);  
    std::cout << "好的做法（鎖外計算）：" << goodTime << " ms" << std::endl;  
    double betterTime = runTest(betterApproach, numThreads, totalWork);  
    std::cout << "更好做法（批次累加）：" << betterTime << " ms" << std::endl;
```



```

std::cout << std::endl;
std::cout << "加速比 (差 vs 好)：" << badTime / goodTime << "x" << std::endl;
std::cout << "加速比 (差 vs 更好)：" << badTime / betterTime << "x" <<
std::endl;
return 0;
}

```

典型輸出

=== 臨界區段大小對效能的影響 ===

執行緒數：4

總工作量：1000

差的做法（鎖內計算）：156.78 ms

好的做法（鎖外計算）：42.35 ms

更好做法（批次累加）：38.92 ms

加速比（差 vs 好）：3.70x

加速比（差 vs 更好）：4.03x

六、臨界區段內應避免的操作

臨界區段內應避免的操作
X 避免在臨界區段內進行：
1. I/O 操作 - 檔案讀寫 - 網路通訊 - 控制台輸出（除錯用途除外）
2. 阻塞等待 - sleep / wait - 等待其他鎖（死結風險） - 等待條件變數

```
| |
| 3. 耗時計算 |
| • 複雜演算法 |
| • 大量數學運算 |
| • 字串處理 |
| |
| 4. 記憶體配置 |
| • new / malloc |
| • 容器擴展（可能觸發重新配置） |
| |
| 5. 呼叫未知函式 |
| • 回呼函式 |
| • 虛擬函式 |
| • 使用者提供的函式 |
| |
```

範例：避免在臨界區段內 I/O

```
// 檔案：lesson_5_6_avoid_io_in_lock.cpp
// 說明：避免在臨界區段內進行 I/O

#include <iostream>
#include <thread>
#include <mutex>
#include <sstream>
#include <vector>

std::mutex mtx;
std::vector<std::string> logs;

// 🧠 差的做法：在鎖內進行 I/O
void badLog(const std::string& message) {
    std::lock_guard<std::mutex> lock(mtx);
    std::cout << message << std::endl; // 🧠 I/O 在鎖內！
}

// ✓ 好的做法：只保護共享資料
void goodLog(const std::string& message) {
    {
```

```

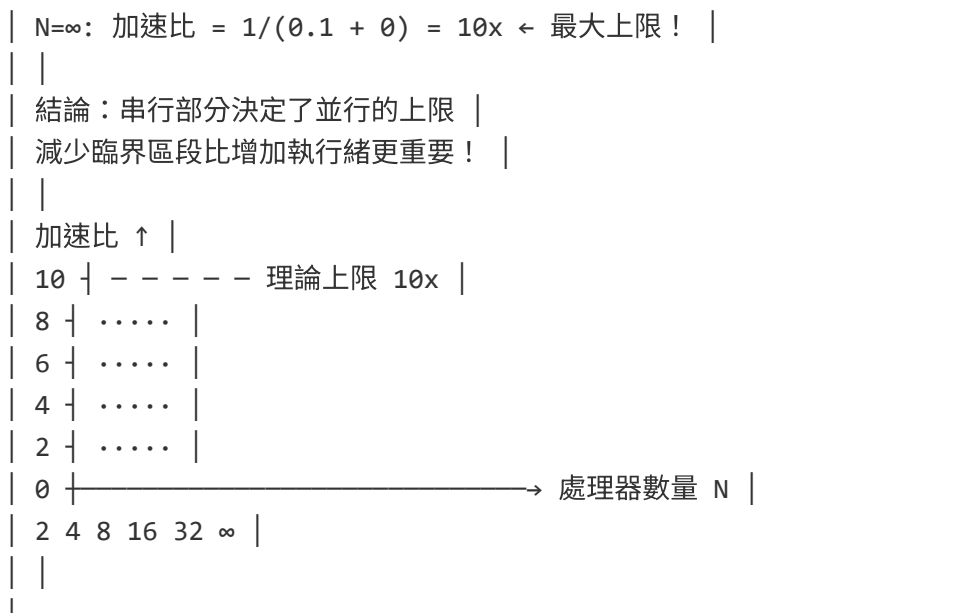
std::lock_guard<std::mutex> lock(mtx);
logs.push_back(message); // ✓ 只保護共享容器
}
// I/O 在鎖外（或由專門的執行緒處理）
}

// ✓ 更好的做法：批次輸出
void flushLogs() {
std::vector<std::string> localLogs;
{
std::lock_guard<std::mutex> lock(mtx);
localLogs.swap(logs); // 快速交換
}
// 在鎖外進行 I/O
for (const auto& log : localLogs) {
std::cout << log << std::endl;
}
}

```

七、Amdahl 定律

Amdahl 定律	
	加速比 = $1 / (S + P/N)$
	S = 程式中串行部分的比例
	P = 程式中可並行部分的比例 (P = 1 - S)
	N = 處理器數量
	範例：假設程式有 10% 必須串行（如鎖保護的部分）
	N=2: 加速比 = $1/(0.1 + 0.9/2) = 1.82x$
	N=4: 加速比 = $1/(0.1 + 0.9/4) = 3.08x$
	N=8: 加速比 = $1/(0.1 + 0.9/8) = 4.71x$



八、效能優化策略總結

效能優化策略總結
策略 1：減少臨界區段大小
- 只鎖定真正需要保護的操作 - 將計算移到鎖外 - 使用本地變數累加後再更新共享資料
策略 2：降低鎖競爭
- 使用細粒度鎖 - 分離讀寫（讀寫鎖，第七階段學習） - 資料分區，每個分區一個鎖
策略 3：減少鎖的使用
- thread_local 變數避免共享 - 原子操作替代簡單的鎖（第二十階段學習）

• 無鎖演算法（第二十二階段學習）	
策略 4：正確選擇鎖類型	
• 讀多寫少：shared_mutex	
• 短臨界區段：自旋鎖	
• 需要超時：timed_mutex	

九、實際案例：優化前後對比

// 檔案：lesson_5_6_optimization_example.cpp
 // 說明：完整的優化前後對比

```
#include <iostream>
#include <thread>
#include <mutex>
#include <vector>
#include <chrono>
#include <numeric>
#include <iomanip>

const int DATA_SIZE = 100000;
const int NUM_THREADS = 4;

// ===== 優化前：單一全域鎖 =====
class BeforeOptimization {
private:
    std::mutex mtx;
    std::vector<int> data;
    long long sum = 0;
public:
    BeforeOptimization() : data(DATA_SIZE) {
        std::iota(data.begin(), data.end(), 1);
    }
    void processRange(int start, int end) {
        for (int i = start; i < end; ++i) {
```

```

std::lock_guard<std::mutex> lock(mtx); // 每次迭代都鎖
sum += data[i] * data[i];
}
}
long long getSum() const { return sum; }
void reset() { sum = 0; }
};

// ===== 優化後：本地累加 + 最後合併 =====
class AfterOptimization {
private:
    std::mutex mtx;
    std::vector<int> data;
    long long sum = 0;
public:
    AfterOptimization() : data(DATA_SIZE) {
        std::iota(data.begin(), data.end(), 1);
    }
    void processRange(int start, int end) {
        long long localSum = 0; // 本地累加
        for (int i = start; i < end; ++i) {
            localSum += data[i] * data[i]; // 無鎖計算
        }
        std::lock_guard<std::mutex> lock(mtx); // 只鎖一次
        sum += localSum;
    }
    long long getSum() const { return sum; }
    void reset() { sum = 0; }
};

template<typename T>
double runTest(T& processor) {
    processor.reset();
    int rangeSize = DATA_SIZE / NUM_THREADS;
    std::vector<std::thread> threads;
    auto start = std::chrono::high_resolution_clock::now();
    for (int i = 0; i < NUM_THREADS; ++i) {
        int rangeStart = i * rangeSize;
        int rangeEnd = (i == NUM_THREADS - 1) ? DATA_SIZE : (i + 1) * rangeSize;
        threads.emplace_back(&T::processRange, &processor, rangeStart, rangeEnd);
    }
}

```

```

for (auto& t : threads) {
    t.join();
}
auto end = std::chrono::high_resolution_clock::now();
return std::chrono::duration<double, std::milli>(end - start).count();
}

int main() {
    BeforeOptimization before;
    AfterOptimization after;
    std::cout << "=== 優化前後效能對比 ===" << std::endl;
    std::cout << "資料大小：" << DATA_SIZE << std::endl;
    std::cout << "執行緒數：" << NUM_THREADS << std::endl << std::endl;
    // 預熱
    runTest(before);
    runTest(after);
    // 正式測試
    double beforeTime = runTest(before);
    double afterTime = runTest(after);
    std::cout << std::fixed << std::setprecision(2);
    std::cout << "優化前時間：" << beforeTime << " ms" << std::endl;
    std::cout << "優化後時間：" << afterTime << " ms" << std::endl;
    std::cout << "加速比：" << beforeTime / afterTime << "x" << std::endl;
    std::cout << std::endl;
    std::cout << "結果驗證：" << std::endl;
    std::cout << "優化前 sum = " << before.getSum() << std::endl;
    std::cout << "優化後 sum = " << after.getSum() << std::endl;
    std::cout << "結果一致：" << (before.getSum() == after.getSum()) ? "√" : "X" <<
    std::endl;
    return 0;
}

```

典型輸出

```

=== 優化前後效能對比 ===
資料大小：100000
執行緒數：4

```

```

優化前時間：245.67 ms
優化後時間：1.23 ms

```

加速比：199.73x

結果驗證：

優化前 sum = 333338333350000

優化後 sum = 333338333350000

結果一致：✓

十、效能檢查清單

效能優化檢查清單	
☐	臨界區段是否盡可能小？
☐	是否有計算可以移到鎖外？
☐	是否可以使用本地變數累加？
☐	臨界區段內是否有 I/O 操作？
☐	臨界區段內是否有阻塞等待？
☐	是否可以使用細粒度鎖？
☐	是否可以使用讀寫鎖？
☐	是否可以用 <code>thread_local</code> 避免共享？
☐	是否可以用原子操作替代鎖？
☐	是否測量了實際的效能影響？

十一、本課重點回顧

1. 鎖不是免費的：無競爭約 20-50 ns，有競爭可達微秒級
2. 競爭越多越慢：高競爭場景下多執行緒比單執行緒還慢
3. 細粒度鎖提升並行度：但增加複雜度和死結風險
4. 減少臨界區段大小：將計算移到鎖外，使用本地累加
5. 避免在臨界區段內：I/O、阻塞等待、耗時計算
6. Amdahl 定律：串行部分決定並行的上限
7. 測量優先：優化前先測量，避免過早優化

第五階段完成！

恭喜你完成了「互斥鎖基礎」階段！你已經學會：

-  `std::mutex` 的基本操作（lock/unlock/try_lock）
-  互斥鎖的內部工作原理
-  常見錯誤與避免方法
-  執行緒安全類別的設計
-  效能考量與優化策略

下一階段預告

第六階段：RAII 鎖管理器 將學習：

課程	主題
6.1	<code>std::lock_guard</code> 基礎
6.2	<code>std::unique_lock</code> 入門
6.3	<code>unique_lock</code> 進階操作
6.4	<code>std::scoped_lock</code> (C++17)
6.5	鎖管理器的選擇策略
6.6	自訂鎖管理器

這些 RAII 工具將讓你的鎖管理更安全、更優雅！

準備好進入第六階段嗎？

 Untitled Attachment